

Definition of AI:

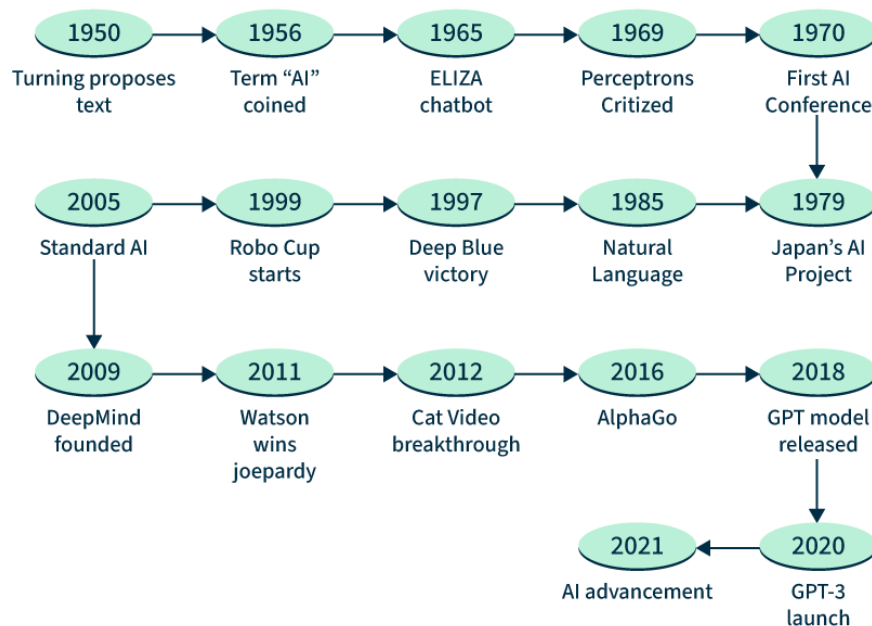
1. Artificial Intelligence (AI) refers to the simulation of human intelligence in machines that are programmed to think, reason, and learn like humans. AI systems use algorithms and vast amounts of data to recognize patterns, make decisions, and improve their performance over time.
2. Artificial intelligence (AI) refers to computer systems capable of performing complex tasks that historically only humans could do, such as reasoning, making decisions, or solving problems. Today, the term "AI" describes a wide range of technologies that power many of the services and goods we use every day – from apps that recommend TV shows to chatbots that provide customer support in real time.
3. Artificial Intelligence (AI) refers to the technology that allows machines and computers to replicate human intelligence.
4. Artificial Intelligence (AI) operates on a core set of concepts and technologies that enable machines to perform tasks that typically require human intelligence.

History of AI:

- 1950 : In 1950 saw the publication of Alan Turing's work , "Computing Machinery and Intelligence " which introduced the Turing Test—a measure of computer intelligence.
- 1956: A significant turning point in AI research occurs in 1956 when John McCarthy first uses the phrase "Artificial Intelligence" at the Dartmouth Workshop.
- 1950s–1960s: The goal of early artificial intelligence (AI) research was to encode human knowledge into computer programs through the use of symbolic reasoning, and logic-based environments.
- 1970: The 1970s witnessed the development of expert systems , which were intended to capture the knowledge of experts in a variety of domains. Data Scientists created rule-based systems that could use pre-established guidelines to address certain issues.
- 1990s: A worldview move towards machine learning approaches happens.
- 2000s-2010s: Profound learning, a subset of machine learning imitating the human brain's structure and work , came to the cutting edge.

- 1642-1936: Charlie's Analytical Engine
- 1834: Tom Stendage: wrote research paper on thinking machine
- 1840: Joseph Faber: first talking machine named Euphonia
- 1921: C Zech author-Karel Capek: R.U.R[Rossum's Universal Robots] It was a science fiction play.
- 1950: Year of Turing Test : Alan Turing [Computing Machinery and Intelligence]
- 1957: Neweel and Simon : They predicted that one day computers will beat human beings in chess.
- 1997: Deep Blue developed by IBM computers: Play Chess ,The game was of 6 match.- first match was played in Philadelphia and the score was 4(garry Kasparov)-2(deep blue)-second match was played in New York and score was 3(deep blue)-2. (1 draw)

Evolution of AI



AI Application Area:

Artificial Intelligence (AI) has become a transformative force across various industries, enhancing efficiency, accuracy, and innovation. Here are some of the key applications of AI:

1. Game Plan:

Reasons: -easy to understand.

-well defined set of rules.

-structured in a way or nature of game is not ambiguous (confusing).

-no financial or ethical burden.

-no expert knowledge is required.

-easy to generate state space.

2. Automated Reasoning and Theorem Proving:-

It is a subfield of automated reasoning and mathematical logic dealing with proving mathematical theorems by computer programs.

3. Natural Language Processing:-

Understanding: machines understand human language.

- Translation: the language converts 1 medium to 2 medium.

- Generation: subtype of NLP

4. Robotics: - A Remotely Operated Vehicle(ROV) is an unoccupied underwater robot

that is connected to a ship by a series of cables.

- cables transmit command and control signals between the operator and the ROV.

- Fully autonomous is a robot that acts without resources to human control.

E.g. Roomba- Semi autonomous is a term for automation that can make decisions and

perform action without direction i.e. they need humans in certain situations.

- Three types of robots: Land, Water, Ariel.

5. Expert Systems:- has knowledge and after the match they take decisions.

- Specific and domain knowledge system should open to learn.- example of expert system:

MYCIN:- It takes a sample of a blood and finds the bacteria that causes the disease in the body.

DENDRAL:- It is a chemical analysis expert system used in organic chemistry to detect unknown organic molecules.

PROSPECTOR:- It was used by geologists to find minerals at particular areas by taking samples.

6. Neural Networks: (Part of deep learning)

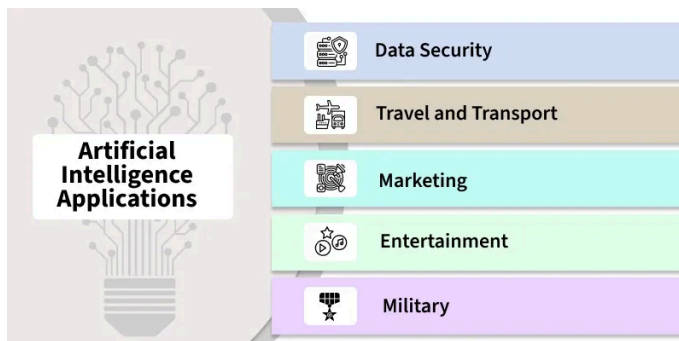
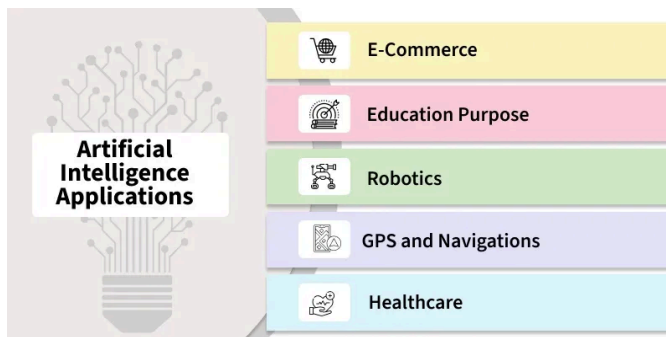
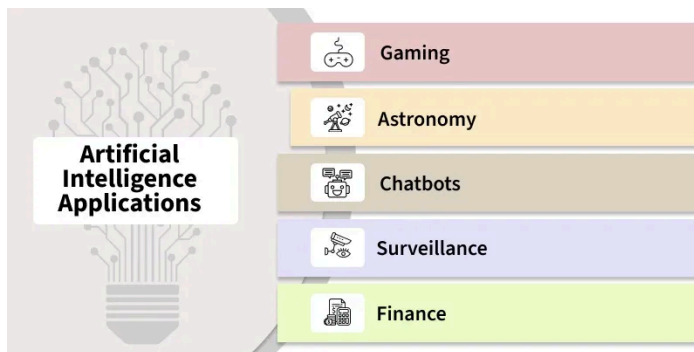
Neural Network is also known as artificial neural network (ANNs).

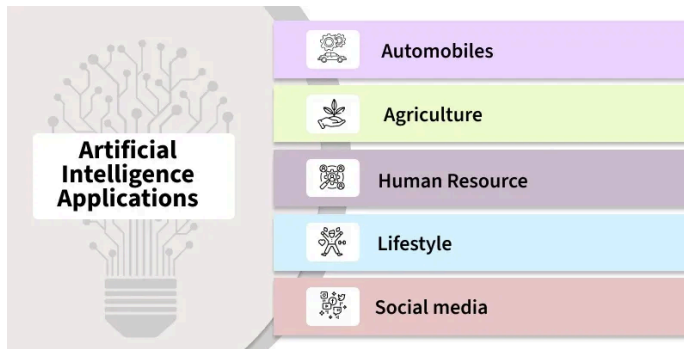
ANN is based on a collection of connected units or nodes called artificial neurons, which loosely model the neurons in a biological brain.

Biological neural network (BNN) is a network of neurons that are connected together by axons and dendrites.

The connections between neurons are made by synapses.

7. Other areas where currently AI is being used





Task Domain of AI:

The domain of AI is classified into different task categories such as Mundane/General tasks, Formal tasks and Expert tasks.

Mundane tasks are the ones that we (the humans) do on a regular basis without any special training such as computer vision, speech recognition, Natural language processing, generation and translation etc. Common sense, reasoning and planning are the common characteristics of these tasks.

Formal tasks - are the ones where there is an application of formal logic, some learning etc. Verifications, Theorem proving etc are the common characteristics. Games such as Chess Checkers, Go etc are classified in these tasks.

Expert task which comes under functional expert domain such as engineering, fault finding, manufacturing planning, medical diagnosis etc.

Expert tasks- Tasks which require high analytical and thinking skills, a job only a professional can do.

Task Domains of Artificial Intelligence		
Mundane (Ordinary) Tasks	Formal Tasks	Expert Tasks
Perception • Computer Vision • Speech, Voice	• Mathematics • Geometry • Logic • Integration and Differentiation	• Engineering • Fault Finding • Manufacturing • Monitoring
Natural Language Processing • Understanding • Language Generation • Language Translation	Games • Go • Chess (Deep Blue) • Checkers	Scientific Analysis
Common Sense	Verification	Financial Analysis
Reasoning	Theorem Proving	Medical Diagnosis
Planning		Creativity
Robotics • Locomotive		

Characteristics of AI:

- It is voluminous (size).
- It is constantly changing.
- It is hard to characterize.
- Its rules will change based on data.
- It is very dynamic.
- Choice of intelligence.

The attributes of AI can be described as its abilities. These characteristics enable AI to be much more than just a computer and permit AI to learn, comprehend, and adapt to its surroundings.

1. Learning Capability

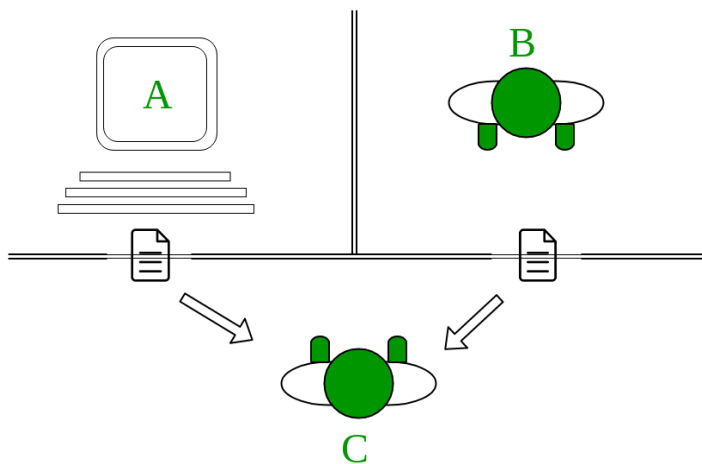
Example of Learning in AI Applications: Use virtual assistants such as Siri and Alexa. In time, they'll understand your preferences as well as pattern of speech. This makes their answers more precise and specific to your preferences.

2. Problem Solving
3. Reasoning
4. Perception
5. Adaptability
6. Automation
7. Decision making
8. Efficiency
9. Self Correction

Turing Test:

- The art of creating machines that perform functions requiring intelligence when performed by people; that is the study of how to make computers do things which, at the moment, people do better.
- Focus is on action, and not intelligent behavior centered around the representation of the world
- Example: Turing Test 3 rooms contain: a person, a computer and an interrogator.
- The interrogator can communicate with the other 2 by teletype (to avoid the machine imitating the appearance of the person) .
- The interrogator tries to determine which the person is and which the machine is.
- The machine tries to fool the interrogator to believe that it is the human, and the person also tries to convince the interrogator that it is the human.
- If the machine succeeds in fooling the interrogator, then conclude that the machine is intelligent.

Watch the video <https://www.youtube.com/watch?v=3wLqsRLvV-c&t=9s>



Defining AI Techniques:

AI techniques refer to methods and algorithms that are used to create smart systems that perform tasks requiring human-like intelligence. Some of these techniques are Machine Learning, Natural Language Processing, Computer Vision and others. These AI techniques use the knowledge efficiently in such a way that –

- It should be perceivable by the people who provide it.
- It should be easily modifiable to correct errors.
- Elevate the speed of execution of the complex program it is equipped with.

AI technique is a method that exploits knowledge(using knowledge in a strategic way) that should be represented in such a way:

- The knowledge captures generalization
- It can be modified to decrease error and to adopt the changes in the environment.

E.g. tic tac toe

Characteristics of AI techniques:

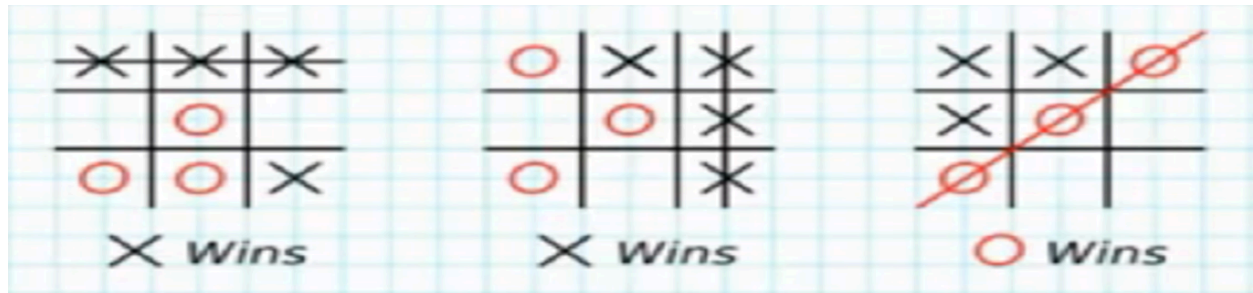
1. It can be used in many situations even if it is not totally accurate or correct.
2. It can be used to overcome its own sheer-bulk by narrowing the range of possibilities.

TIC TAC TOE (Strategy-1): Blind Method / Random method

Start with **center**, then **corner**, then sides – simple, static rule.

It's a paper and pencil game of two players X and O, who choose to mark a space on a grid of 3×3.

The game is won by the player who succeeds in putting three of their marks in a horizontal line, vertical line, or diagonal line.



This game is played by two players – One is a human and the other is a computer.

The objective is to write a computer program in such a way that the computer wins most of the time. Three approaches are presented to play this game which increases in

1. Complexity
2. Use of generalization
3. Clarity of their knowledge
4. Extensibility of their approach

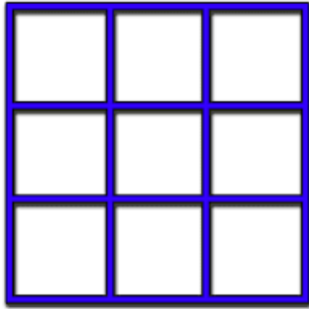
These approaches will move towards being representations of what we will call AI techniques.

Assume , Player 1 – X Player 2 – O

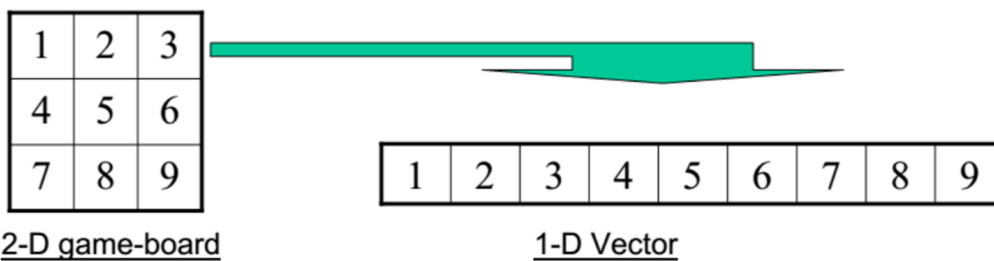
So, a player who gets 3 consecutive marks first, he will win the game.

Let's have a discussion about how a board's data structure looks and how the Tic Tac Toe algorithm works.

Board Data Structure:



Each square is numbered, which starts from 1 to 9 like the following image.



Data Structures: Board and Move Table

Consider a Board having nine elements vectors. Each element will contain

- 0 for blank
- 1 indicating X player move
- 2 indicating O player move

The computer may play as an X or O player.

The first player is always X.

Move Table: It is a vector of 3^9 elements, each element of which is a nine-element vector representing board position.

Total of 3^9 (19683) elements in the move table. Sample Move Table looks like as shown below:

Index	Current Board position	New Board Position
0	000000000	000010000
1	000000001	020000001
2	000000002	000100002
3	000000010	002000010
....		
19682		

Algorithm:

Step 1: View the vector (board) as a ternary number and convert it to its corresponding decimal number.

Step 2: Use the computed number as an index in the move table and access the vector stored there.

Step 3: The vector selected in step 2 represents the way the board will look after the move that should be made. So set the board equal to that vector.

This program is very efficient in terms of time. And, in theory, it could play an optimal game of tic-tac-toe.

But it has several disadvantages:

- Takes lot of space
- Should specify entries in moveTable
- Movetable entries are determined and entered without any errors

Advantages	Disadvantages
You can play all moves.	Space requirement is more
Optimized move	Time required to prepare move table
Possibilities of update on move table	It is not scalable
Keep an eye on opponent move	Hard work is required to prepare such a table.

TIC TAC TOE (Strategy-2): Numeric Method

Start with the corner, aim to create forks – better offensive potential.

Board : A nine element vector representing the board, as described for Program 1. But instead of using the numbers 0,1, or 2 in each element, we store 2 (indicating blank), 3 (indicating X), or 5 (indicating O).

Turn : An integer indicating which move of the game is about to be played; 1 indicates the first move, 9 indicates the last .

The Algorithm : The main algorithm uses three subprocedures:

Make 2: Returns 5 if the center square of the board is blank, that is, if Board[5] = 2. Otherwise, this function returns any blank non corner square (2, 4, 6, or 8).

Posswin(p) : Returns 0 if player p cannot win on his next move; otherwise, it returns the number of the square that constitutes a winning move. This function will enable the program both to win and to block the opponent's win. Posswin operates by checking, one at a time, each of the rows, columns, and diagonals. Because of the way values are numbered, it can test an entire row (column or diagonal) to see if it is a possible win by multiplying the values of its squares together. If the product is 18 (3 x 3x 2), then X can win. If the product is 50(5 x 5 x 2), then O can win. If we find a winning row, we determine which element is blank, and return the number of that square.

Go(n) : Makes a move in square n. This procedure sets Board[n] to 3 if Turn is odd, or 5 if Turn is even. It also increments Turn by one.

Turn=1 (X)	Go(1) (upper left corner).
Turn=2 (O)	If board[5] is blank, Go(5), else Go(1).
Turn=3 (X)	If board[9] is blank, Go(9), else Go(3).
Turn=4 (O)	If Posswin(X) is not 0, then Go(Posswin(X)) [i.e. block opponent's win],else Go(make2).
Turn=5 (X)	If Posswin(X) is not 0, then Go(Posswin(X)) [i.e. win] else Posswin(O) is not 0, then Go(Posswin(O))[i.e. block win], else if board[7] is blank , then Go(7) , else Go(3).
Turn=6 (O)	If Posswin(O) is not 0, then Go(Posswin(O)) [i.e. win] else Posswin(X) is not 0, then Go(Posswin(X))[i.e. block win], Else Go(Make2).

Turn=7 (X)	If Posswin(X) is not 0, then Go(Posswin(X)) [i.e. win] else Posswin(O) is not 0, then Go(Posswin(O))[i.e. block win], else go anywhere that is blank.
Turn=8 (O)	If Posswin(O) is not 0, then Go(Posswin(O)) [i.e. win] else Posswin(X) is not 0, then Go(Posswin(X))[i.e. block win], Else go anywhere that is blank.
Turn=9 (X)	Same as turn=7.

- This strategy is not efficient in terms of time (as it has to check several conditions before making a move), more efficient in terms of space.
- Easy to understand the program's strategy or change strategy if required.
- Still it cannot generalize to 3 Dimensions.

TIC TAC TOE-(Strategy-2') / Even- odd Algorithm

Maps the 3x3 board to a 3x3 magic square (each row/col/diag sums to 15). Used for detecting win/loss conditions mathematically.

Algorithm

This program is identical to strategy 2 except for one change in the representation of the board. We again represent the board as a nine-element vector, but this time we assign board positions to vector elements as follows:

8	1	6
3	5	7
4	9	2

Notice that this numbering of the board produces a magic square: all the rows, columns, and diagonals sum up to 15.

This means that we can simplify the process of checking for a possible win. In addition to marking the board as moves are made, we keep a list, for each player, of the squares in which he or she has played. To check for a possible win for one player, we consider each pair of squares owned by that player and compute the difference between 15 and the sum of the two squares.

If this difference is not positive or if it is greater than 9, then the original two squares were not collinear and so can be ignored. Otherwise, if the square representing the difference is blank, a move there will produce a win.

Since no player can have more than four squares at a time, there will be many fewer squares examined using this scheme than there were using the more straightforward approach of strategy 2.

This shows how the choice of representation can have a major impact on the efficiency of a problem-solving program.

TIC TAC TOE (strategy-3): Min Max Algorithm

A recursive AI strategy that simulates **all possible moves** and picks the best outcome by assuming the opponent plays optimally.

Board Position: A structure containing a nine-element vector representing the board, a list of board positions that could result from the next move, and a number representing an estimate of how likely the board position is to lead to an ultimate win for the player to move.

The Algorithm :

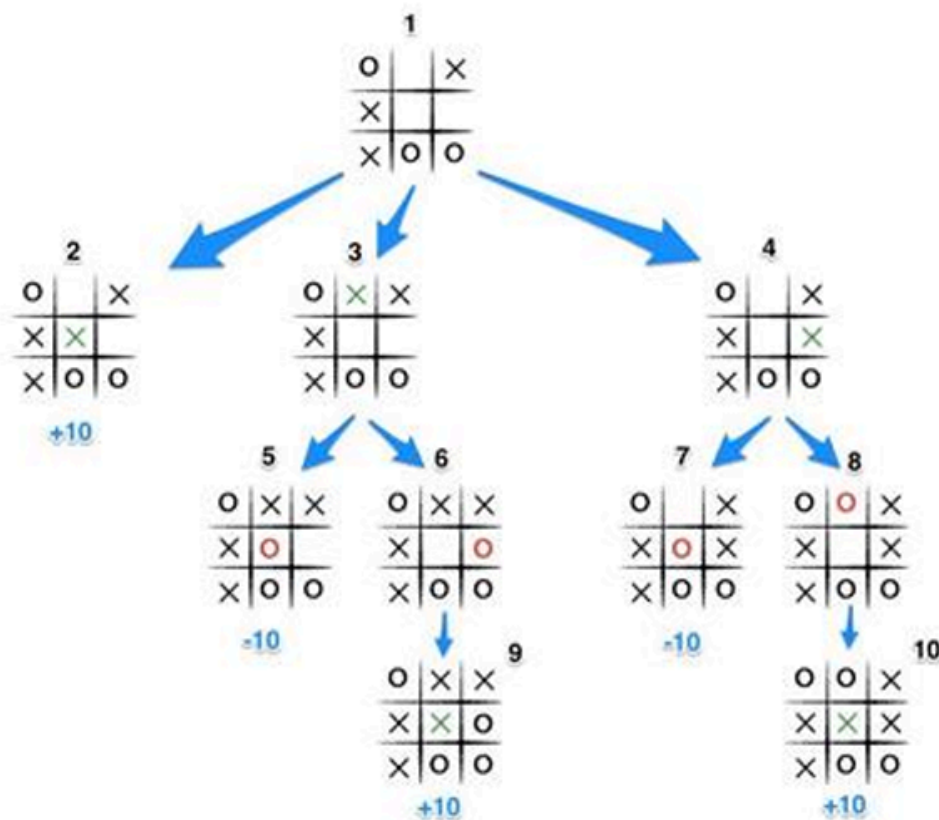
To decide on the next move, look ahead at the board positions that result from each possible move. Decide which position is best (as described below), make the move that leads to that position, and assign the rating of that best move to the current position.

To decide which of a set of board positions is best, do the following for each of them:

1. See if it is a win. If so, call it the best by giving it the highest possible rating.
2. Otherwise, consider all the moves the opponent could make next. See which of them is worst for us (by recursively calling this procedure). Assume the opponent will make that move. Whatever rating that move has, assign it to the node we are considering.
3. The best node is then the one with the highest rating.

This algorithm will look ahead at various sequences of moves in order to find a sequence that leads to a win. It attempts to maximize the likelihood of winning, while assuming that the opponent will try to minimize that likelihood.

For eg:



Benefits of strategy3:

- Min-Max is the perfect play game.
- It either wins or draws and never loses.
- While strategy 1 and 2 the player can lose the game if the opponent plays well.
- Minmax adapts to the current board state at every turn.
- Strategy 1 and 2 follow a fixed move table.
- Strategy 1 and 2 depends on predefined logic or values.
- Minmax uses game tree traversal and not pre-computed tricks.
- Minmax can be applied to other games like chess, connect four etc.
- Magic square is only for Tic Tac Toe games and doesn't scale.
- Minmax assumes the opponent will also play optimally.
- Other strategies don't consider the opponent's skill/behaviour.
- Minmax is a standard in AI game Development

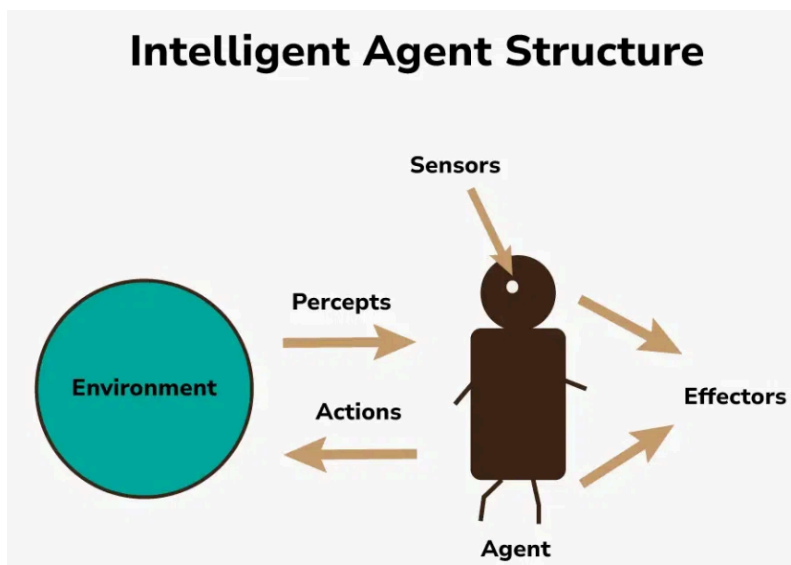
Intelligent Agents:

An intelligent agent is an entity or system that perceives its environment through sensors and acts upon that environment through actuators in such a way that maximizes the chance of achieving a goal. An agent can be software based, hardware based or both.

Eg: Thermostat , Google Assistant , spam filter in mail, self checkout machine in supermarket, Roomba robotic vacuum cleaner .

Characteristics of an intelligent agent:

- Autonomous: Operates without human intervention
- Perception: Observes the environment through sensors
- Reasoning: Process the signal and make decisions based on goals
- Learning: Improves performance based on past experiences.
- Action: Take action to influence the environment.



Basically an agent is an entity that senses and acts.

AI based definition of agent: An agent is a computational system that is situated in some environment and is capable of autonomous actions to achieve its goals.

Types of Agents:

- Human agent: Eyes (sensors), hands/legs (actuators), brain (processor).
- Robotic agent: Cameras, IR sensors → motors, speakers.
- Software agent: Inputs from code/files → Outputs to screen, API, internet.

Sensors: They are tools that an AI agent uses to perceive their environment. They can be anything physical like cameras, microphones, temperature sensors or a software sensor that reads data from files.

Actuators: They are tools that AI agents use to interact with their environment through some actions. They can be any physical actuators like wheels, motors, robotic hands, or computer screens or they can be software actuators that send messages.

Effectors: It takes instructions from decision making mechanisms and translates them into actions and these actions are performed through actuators.

Rationality: When an agent behaves or acts in a way that is **logically sound and goal-oriented** based on available information then we call such agents as rational.

The factor or the parameter that determines that the agent acts rationally is called PEAS.

The rationality of an agent is measured by its performance measure.
Rationality can be judged on the basis of following points:-

- Performance measure which defines the success criterion.
- Agent prior knowledge of its environment.
- Best possible actions that an agent can perform.

PEAS stands for performance measure, environment, actuators and sensors. It is a framework that is used to describe an AI agent. It's a structured approach to design and understand AI systems. In designing an agent, the first step must always be to specify the task environment as fully as possible..

The rationality of an agent is measured by its performance measure. Rationality can be judged on the basis of following points:

- Performance measure which defines the success criterion.
- Agent prior knowledge of its environment.
- Best possible actions that an agent can perform.

Nature of Environment: The **environment** is everything external to the agent that it interacts with. The **nature of the environment** plays a pivotal role in how an agent behaves, what kind of decisions it can make, and how intelligent it needs to be.

PEAS stands for performance measure, environment, actuators and sensors. It is a framework that is used to describe an AI agent. It's a structured approach to design and understand AI systems.

1. **Performance measure:** Performance measure is a criteria that measures the success of the agent. It is used to evaluate how well the agent is achieving its goal.
2. **Environment:** The environment represents the domain or context in which the agent operates and interacts. This can range from physical spaces like rooms to virtual environments such as game worlds or online platforms like the internet.
3. **Actuators:** Actuators are the mechanisms through which the AI agent performs actions or interacts with its environment to achieve its goals.

These can include physical actuators like motors and robotic hands, as well as digital actuators like computer screens and text-to-speech converters.

4. **Sensors:** Sensors enable the AI agent to gather information from its environment, providing data that informs its decision-making process and actions. These sensors can capture various environmental parameters such as temperature, sound, movement, or visual input. Examples of sensors include cameras, microphones, temperature sensors, and motion sensors.

Eg: Agent under PEAS Measure

Agent Name	Robotic Vacuum cleaner
Performance Measure	Cleanliness, battery usage, time efficiency, area coverage
Environment	Home interior(floor, furniture, other obstacle)
Actuators	Wheels, brushes, vacuum bag
Sensors	Infrared sensors, bump sensors, dirt detectors, gyroscope etc

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	healthy patient, costs, lawsuits	patient, hospital, stuff	display questions, tests, diagnoses, treatments, referrals	keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	correct image categorization	downlink from orbiting satellite	display categorization of scene	color pixel arrays
Part-picking robot	percentage of parts in correct bins	conveyor belt with parts, bins	jointed arm and hand	camera, joint angle sensors
Refinery controller	purity, yield, safety	refinery, operators	valves pumps, heaters displays	temperature, pressure, chemical sensors
Interactive English tutor	student's score on test	set of students, testing agency	display exercises, suggestions, corrections	keyboard entry

Some more example of agent for your practice:

1. Smart Thermostat (eg: Nest)
2. Google map Navigation Assistant
3. Email Spam filter
4. Industrial Robot Arm
5. Self Driving Car
6. Face Unlock on SmartPhones
7. ATM Machine
8. Smartwatch Health Tracker.
9. Online Food delivery App.

Types of Environment:

1. **Accessible vs. inaccessible or Fully observable vs Partially**

Observable: If an agent sensor can sense or access the complete state of an environment at each point of time then it is a fully observable environment, else it is partially observable.

Eg: fully observable environments include board games like chess or checkers.

Eg: partially observable environments include driving a car in traffic.

2. **Deterministic vs. Stochastic:** If the next state of the environment is completely determined by the current state and the actions selected by the agents, then we say the environment is deterministic.

Eg: deterministic environments include simple mathematical equations, where the outcome of each operation is precisely defined.

Eg: stochastic environments include games of chance like poker or roulette, where the outcome of each action is influenced by random factors like the shuffle of cards or the spin of a wheel.

3. Episodic vs. nonepisodic: The agent's experience is divided into "episodes." Each episode consists of the agent perceiving and then acting. The quality of its action depends just on the episode itself, because subsequent episodes do not depend on what actions occur in previous episodes. Episodic environments are much simpler because the agent does not need to think ahead.

Eg: Pick and Place robot, which is used to detect defective parts from the conveyor belts. Here, every time the robot(agent) will make the decision on the current part i.e. there is no dependency between current and previous decisions.

Nonepisodic or sequential : The previous decisions can affect all future decisions. The next action of the agent depends on what action he has taken previously and what action he is supposed to take in the future.

Eg: Checkers- Where the previous move can affect all the following moves.

4. Static Vs Dynamic: An environment that keeps constantly changing itself when the agent is up with some action is said to be dynamic.

Eg: A roller coaster ride is dynamic as it is set in motion and the environment keeps changing every instant. (Real world driving)

An idle environment with no change in its state is called a static environment.

Eg: An empty house is static as there's no change in the surroundings when an agent enters.(crossword puzzle)

5. Discrete Vs Continuous: If an environment consists of a finite number of actions that can be deliberated in the environment to obtain the output, it is said to be a discrete environment.

Eg: The game of chess is discrete as it has only a finite number of moves. The number of moves might vary with every game, but still, it's finite.

The environment in which the actions are performed cannot be numbered i.e. is not discrete, is said to be continuous.

Eg: Self-driving cars are an example of continuous environments as their actions are driving, parking, etc. which cannot be numbered. Like drone navigation.

6. Single Agent Vs Multi Agent: An environment consisting of only one agent is said to be a single-agent environment.

Eg: A person left alone in a maze is an example of the single-agent system.

An environment involving more than one agent is a multi-agent environment.

Eg: The game of football is multi-agent as it involves 11 players in each team.

7. Known Vs Unknown: In a known environment, the output for all probable actions is given. Obviously, in case of an unknown environment, for an agent to make a decision, it has to gain knowledge about how the environment works.

Eg: Lab Simulations: Agent knows the rules and consequences of actions.

Eg: Stock market prediction: Agents must learn the rules through experience.

Self-driving taxis operate in a **partially observable, stochastic, sequential, dynamic, continuous, and partially known** environment.

Examples of task environments

Task Environment	Observable	Deterministic	Episodic	Static	Discrete	Agents
Crossword puzzle	Fully	Deterministic	Sequential	Static	Discrete	Single
Chess with a clock	Fully	Strategic	Sequential	Semi	Discrete	Multi
Poker	Partially	Strategic	Sequential	Static	Discrete	Multi
Backgammon	Fully	Stochastic	Sequential	Static	Discrete	Multi
Taxi driving	Partially	Stochastic	Sequential	Dynamic	Continuous	Multi
Medical diagnosis	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Image-analysis	Fully	Deterministic	Episodic	Semi	Continuous	Single
Part-picking robot	Partially	Stochastic	Episodic	Dynamic	Continuous	Single
Refinery controller	Partially	Stochastic	Sequential	Dynamic	Continuous	Single
Interactive English tutor	Partially	Stochastic	Sequential	Dynamic	Discrete	Multi

Figure 2.6 Examples of task environments and their characteristics.

Based on the above, prepare the sheet where you can judge each example based on the types of environment above whether it is present or absent , if present explain the reason.

For eg: Cleaning Robot

Environment Type	present/absent	explanation
------------------	----------------	-------------

Similarly for Online Shopping Recommender System, voice assistant, drone delivery system, ATM Machine.

Structure of Agents:

The job of AI is to design the agent program: a function that implements the agent mapping from percepts to actions. We assume this program will run on some sort of ARCHITECTURE computing device, which we will call the architecture.

agent = architecture + program

The architecture might be a plain computer, or it might include special-purpose hardware for certain tasks, such as processing camera

images or filtering audio input. It might also include software that provides a degree of insulation between the raw computer and the agent program, so that we can program at a higher level. In general, the architecture makes the percepts from the sensors available to the program, runs the program, and feeds the program's action choices to the effectors as they are generated.

Program:

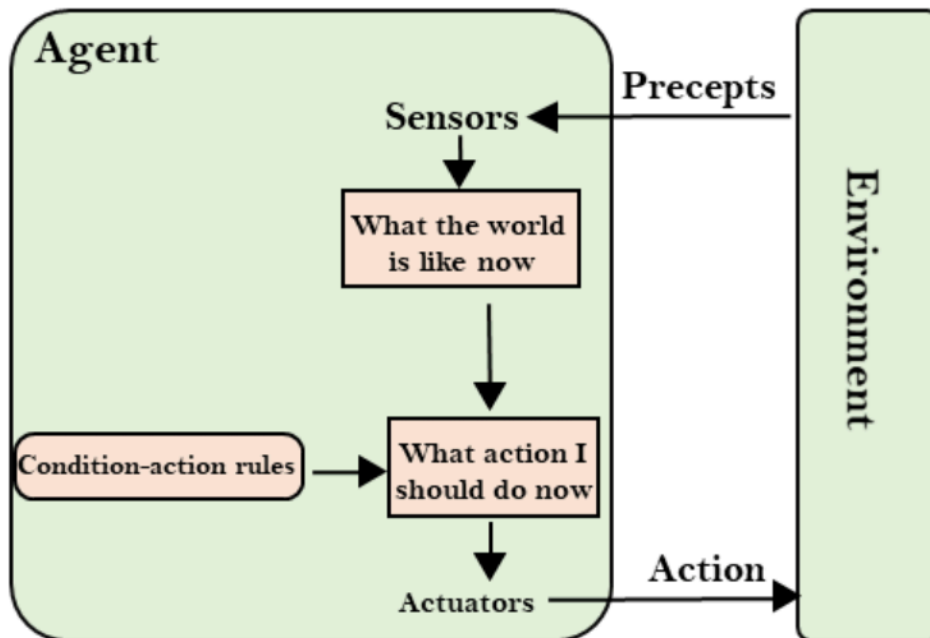
- Intelligent agents accept percepts from an environment and generate actions. The early versions of agent programs will have a very simple form.
- Each will use some internal data structures that will be updated as new percepts arrive.
- These data structures are operated on by the agent's decision-making procedures to generate an action choice, which is then passed to the architecture to be executed

Types of Agents:

1. Simple Reflex Agents
2. Model-Based Reflex Agents
3. Goal-Based Agents
4. Utility-Based Agents

1.Simple Reflex Agents:

- They choose actions only based on the current percept and ignore the rest percept history.
- They are rational only if a correct decision is made only on the basis of current precept.
- Their environment is completely observable.
- It works on preset rules and preprogrammed outcomes and works within those parameters.
- Implementation through condition action rules i.e action is taken only when condition is true.



Eg: Mail Sorting Robot

Environment—conveyor belt/box of letters

Rule— Area based sorting(Ahmedabad, Rajkot, Vadodara etc)

Action— pick and drop in dedicated bins of area.

Function: SIMPLE_REFLEX_AGENT(percept)

Return action

Static:rules;(set of condition action pair)

State←- INTERPRET_INPUT(percept)

Rule←- RULE_MATCH(state,rule)

Action←- RULE_ACTION(rule)

Return action

Limitations: 1. Limited intelligence

1. No knowledge for the non-percept part of the current state.
2. Too big to generate and store rules.
3. Capable of adapting to changes in the environment.
4. Any change in environment ~ rules need to be updated.

Eg: 1. Thermostat

2. Vacuum cleaner

3. Automatic soap dispenser

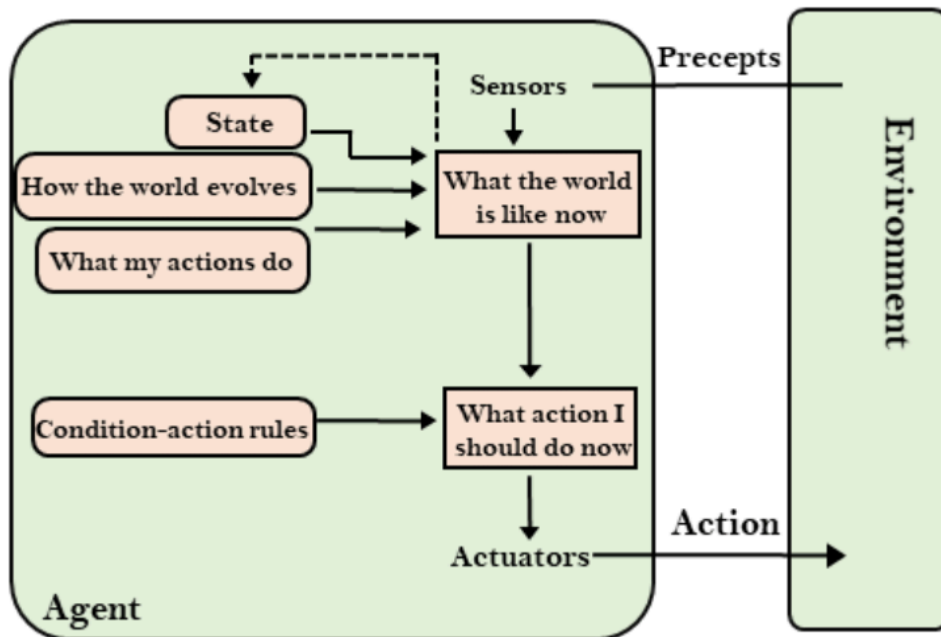
4. Automatic room light

2. Model-based Agents:

The Model-based agent can work in a partially observable environment, and track the situation.

A model-based agent has two important factors:

- Model: It is knowledge about "how things happen in the world," so it is called a Model-based agent.
- Internal State: It is a representation of the current state based on percept history.

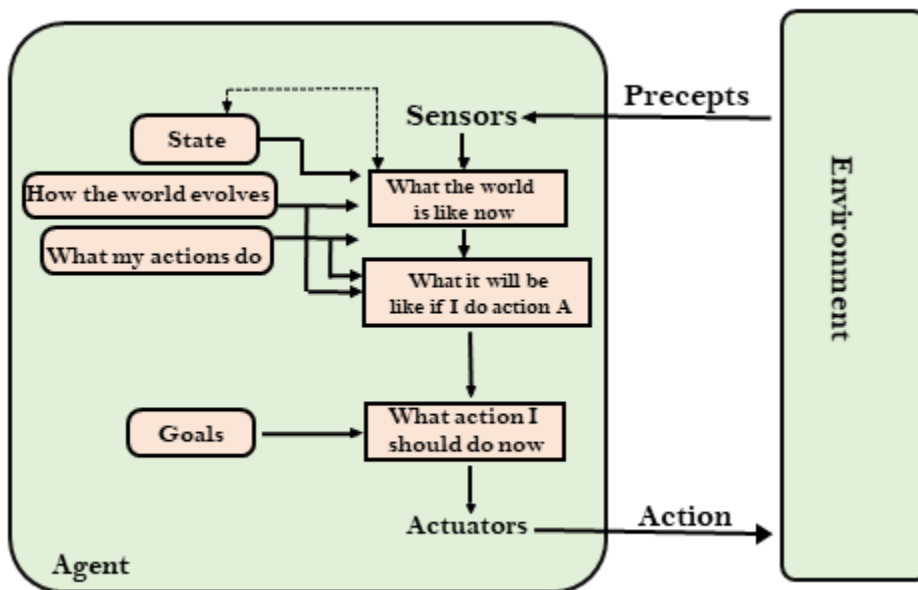


- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:
 - How the world evolves
 - How the agent's action affects the world.

3. Goal Based Agent:

The knowledge of the current state environment is not always sufficient to decide for an agent to decide what to do. The agent needs to know its goal which describes desirable situations. Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.

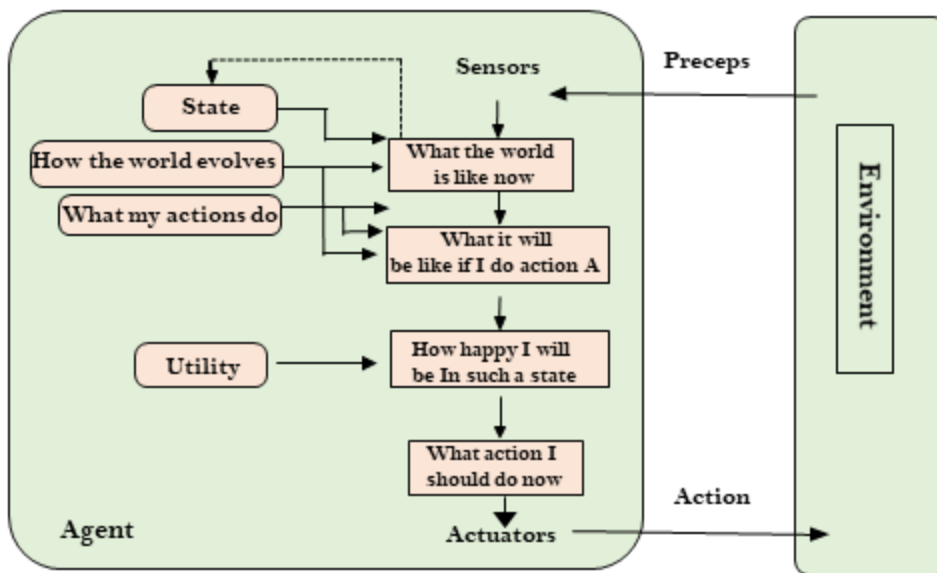
They choose an action so that they can achieve the goal. These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenarios are called searching and planning, which makes an agent proactive.



4. Utility Based Agent:

These agents are similar to the goal-based agent but provide an extra component of utility measurement, which makes them different by providing a measure of success at a given state. Utility-based agents act based not only on goals but also the best way to achieve the goal.

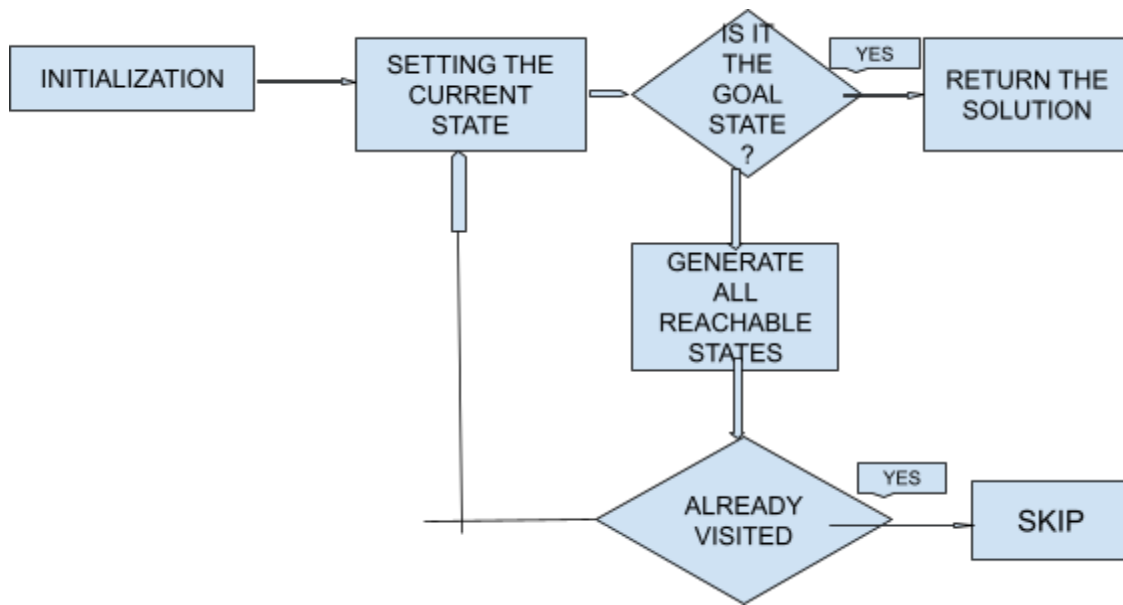
The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action. The utility function maps each state to a real number to check how efficiently each action achieves the goals.



State Space Search:

Defining problems as state space search: State Space Search is used to solve problems by finding different possible states and their transitions. In simple terms it's like finding the best route to your goal by trying different paths.

- A state space is a mathematical representation of a system where each unique state is represented by a point in the state space.
- It is used to represent the current state of a system as well as the possible state that the system can transit into.
- State space is used in the search algorithm to represent the initial state, goal state and current state of the problem.
- Each state in the state space is represented using a set of variables.
 $S = \{s_1, s_2, s_3, s_4, \dots\}$



Example: 8 Puzzle Problem

- It is a sliding puzzle that consists of 8 numbered tiles in a 3x3 grid and one blank space.
- The goal is to arrange tiles from initial state to terminal/goal state by sliding tiles into blank space.
- Let the initial state is

Current State

1		2
3	4	5
6	7	8

Goal State

1	4	2
3	7	5
6		8

The core idea behind state space search is to think of the problem as a graph where:

- **Nodes** represent different states of the problem.
- **Edges** represent transitions or actions that change one state into another.

Terminologies consist of:

- **State:** A specific configuration of the problem.
- **Initial State:** Starting point of the search.
- **Goal State:** Desired end configuration.
- **Transition:** An action that changes one state to another.
- **Path:** A sequence of states connected by transitions.
- **Search Strategy:** Method used to explore the state space.

Steps in State Space Search:

1. **Define State Space :** Identify all possible states and their transitions.
To do this model the problem in a way that includes all relevant configurations and actions.
2. **Pick a search Strategy:** Choose a method for exploring the state space.
Common strategies include: BFS,DFS, Greedy best first search, A* search etc.
3. **Start the search:** Add initial state and begin the search from there.
4. **Extend the nodes:** Using the selected search technique it expands nodes from the start state and helps in generating successor states and adding them to the frontier. If a state matches the goal it retraces the path to the solution and stops the search.
5. **Address state repetition:** Prevent revisiting the same state by tracking visited states helps in avoiding cycles and unnecessary exploration.
6. **End the search:** The search ends when the goal state is found or when all states have been explored without finding a solution.

The efficiency of state space search depends on several key factors:

- **Expansiveness:** Number of new states a given state can generate. More broader problems require finding more possibilities which increases search complexity.
- **Branching Factor:** Average number of successors per state. A higher branching factor makes the search tree wider which increases time and resources required.
- **Depth:** Number of steps from the initial state to the goal state. Deeper trees increase the search time as more states must be explored.
- **Completeness:** A search is complete if it guarantees finding a solution if one exists. It ensures the algorithm will eventually reach the goal.
- **Optimality:** A search is optimal if it guarantees finding the best solution, according to a specific criterion like least cost or shortest path.
- **Time Complexity:** The time required for the search which gets affected by branching factor and depth. More expansive or deeper trees increase the time needed.
- **Space Complexity:** Memory required for the search. It depends on how many states need to be stored at once. Larger branching factors or deeper searches use more memory.

Problem Characteristics:

Since artificial intelligence (AI) is mainly related to the search process, it is important to have some methodology to choose the best possible solution.

To choose an appropriate method for a particular problem first we need to categorize the problem based on the following characteristics.

1. Is the problem decomposable into small sub-problems which are easy to solve?
2. Can solution steps be ignored or undone?
3. Is the universe of the problem predictable?
4. Is a good solution to the problem absolute or relative?
5. Is the solution to the problem a state or a path?
6. What is the role of knowledge in solving a problem using artificial intelligence?
7. Does the task of solving a problem require human interaction?

Production systems and characteristics

A production system in AI is a decision-making model based on *if-then* rules and a shared memory (called working memory) that stores facts. When a rule's "if" condition matches the current facts, its "then" action is triggered, updating or adding new information.

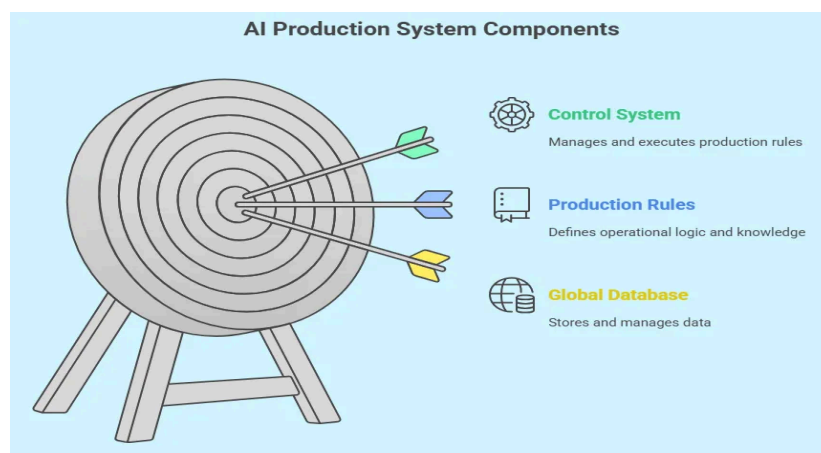
Eg: college library system:

- A college library system checks each returned book's status.
- If the system finds the return is late, it calculates how many days overdue it is.
- A penalty is added to the student's record if the overdue period crosses a certain threshold.
- Additional rules may apply if a user continues returning books late, such as a higher penalty or a warning notice.
- When no more conditions match, like if all books are returned on time the system stops and logs the final state.

Components of a Production System:

Production systems rely on three essential components that shape how rules interact with stored information. These core components are the global database, the production rules, and the control system.

Each serves a distinct purpose together; they determine how facts are updated, which rules are triggered, and how the system ultimately makes its decisions.



1. The Global Database

- It holds the current facts or conditions the system checks against rules.
- When new data arrives, it updates to reflect the latest state of the system.

2. The Production Rules (Knowledge Base)

- These are if-then rules that turn stored data into actions.
- They guide the system from general facts to specific results without extra coding.

3. The Control System (Inference Engine)

- It decides which rule to apply and when to stop processing.
- If multiple rules apply, it chooses the correct order to keep actions smooth.

Types of Production Systems

All Production systems in AI are not constructed in the same manner. Some rely on stored facts about the domain, while others emphasize step-by-step instructions and direct rules. Each strategy has its own advantages, so it's helpful to compare them before choosing the one that best fits your needs.

1. Rule-Based Production System in AI

These systems use if-then rules to make decisions. If a condition is true, the system takes an action or gives a conclusion.

How it works:

- Rules are stored in a knowledge base.
- Facts go into a global database.
- If a rule's "if" matches the facts, the "then" part runs.
- This repeats until no rules apply.

Examples:

- Troubleshooting: Fixing router issues
- Loan Checks: Matching applicant info to bank criteria
- Form Validation: Checking exam form entries

2. Procedural Systems

These systems follow a step-by-step process to solve a problem. Each step depends on the previous one.

How it works:

- Predefined steps guide the process
- It follows the steps in order
- Facts are updated after each step
- Errors may stop or restart the process

Examples:

- Manufacturing: Controlling assembly line stages
- Form Submissions: Guiding users step by step
- Software Installation: Wizards for updates

3. Declarative Systems

They focus on what outcome is needed, not how to get there. The system uses stored facts and relationships to answer queries.

How it works:

- Facts and relations are stored
- Queries are submitted
- If facts match, an answer is given
- The process behind the result isn't shown

Examples:

- Medical Database: Suggesting diseases based on symptoms
- Hardware Matcher: Finding compatible parts
- Campus FAQs: Giving direct answers to common questions

Production System Characteristics

Positive Characteristics:

- **Simplicity:** Easy to understand with clear “if this, then that” logic.
- **Modularity:** Rules work independently, so changes don’t affect the whole system.
- **Knowledge-Focused:** Rules reflect real-world knowledge in plain language.
- **Reactivity:** The system adapts when facts change, making it fit for real-time tasks.

Negative Characteristics:

- **Opacity:** It’s hard to trace which rule caused a result when many rules fire.
- **Inefficiency:** Systems may slow down as they check all rules every time.
- **No Learning:** They don’t learn or update rules unless manually changed.
- **Conflict Handling:** Extra logic is needed when multiple rules match at once.

Categories of a Production System : categories highlight how a system treats new or existing facts during execution. There are four main categories that often come up, each reflecting a distinct way of handling knowledge updates and state changes.

1. Monotonic Production System in AI

A monotonic system never invalidates facts once they’ve been confirmed. This means each new conclusion remains in place, and one rule’s action doesn’t block another from becoming valid later.

As the knowledge grows, the system consistently adds to existing facts rather than reversing them.

2. Partially Commutative Production System in AI

In a partially commutative system, some sequences of rules can swap places without changing the final outcome. Although not all steps are interchangeable,

enough flexibility exists to allow multiple valid paths that converge on the same result.

3. Non-Monotonic Production System in AI

A non-monotonic system allows facts or conclusions to be retracted if they conflict with new information. Rather than storing every outcome permanently, it can drop or revise conclusions when fresh data shows an earlier inference was wrong.

4. Commutative Production System in AI

In a commutative system, the order of rule firing doesn't affect the final outcome, as long as each rule application is allowed. Any valid sequence of steps arrives at the same end state reflecting a highly uniform structure.

Heuristics:

Many traditional search algorithms are used in AI applications. For complex problems, the traditional algorithms are unable to find the solutions within some practical time and space limits. Consequently, many special techniques are developed, using heuristic functions. The algorithms that use heuristic functions are called heuristic algorithms.

- Heuristic algorithms are not really intelligent; they appear to be intelligent because they achieve better performance.
- Heuristic algorithms are more efficient because they take advantage of feedback from the data to direct the search path.
- Uninformed search algorithms or Brute-force algorithms, search through the search space all possible candidates for the solution checking whether each candidate satisfies the problem's statement.
- Informed search algorithms use heuristic functions that are specific to the problem, apply them to guide the search through the search space to try to reduce the amount of time spent in searching.

Breadth First Search

Breadth-First –Search is an uninformed search technique. Consider the state space of a problem that takes the form of a tree. Now, if we search the goal along with each

breadth of the tree, starting from the root and continuing up to the largest depth, we call it breadth-first search.

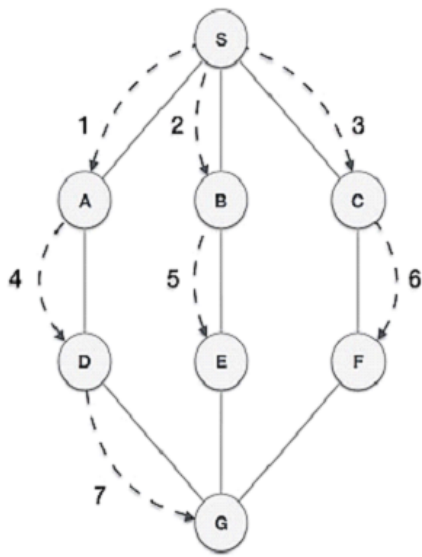
- A Search strategy, in which the highest layer of a decision tree is searched completely before proceeding to the next layer is called Breadth-first search (BFS).
- In this strategy, no viable solutions are omitted and therefore it is guaranteed that an optimal solution is found.
- This strategy is often not feasible when the search space is large.
- Algorithm:
 1. Create a variable called QUEUE and set it to be the starting state.
 2. Loop until a goal state is found or QUEUE is empty, Do
 - a. Remove the first element from the QUEUE and call it VISITED. If the QUEUE is empty, quit.
 - b. For every path each rule can match the state VISITED, Do
 - (i) Apply the rule to generate a new state.
 - (ii) If the new state is a goal state, quit and return to this state.
 - (iii) Otherwise, add the new state to the end of QUEUE.

Advantages :1. Guaranteed to find an optimal solution (in terms of shortest number of steps to reach the goal).
2. Can always find a goal node if one exists (complete).

Disadvantages :1. High storage requirement: exponential with tree depth.

Example: BFS is basically a nodebased algorithm which is used to find the shortest path in the graph between two nodes. BFS moves through all of its nodes which are connected to the individual nodes.

BFS uses the FIFO (First In First Out) principle while using the Queue to find the shortest path. However, BFS is slower and requires a large memory space.

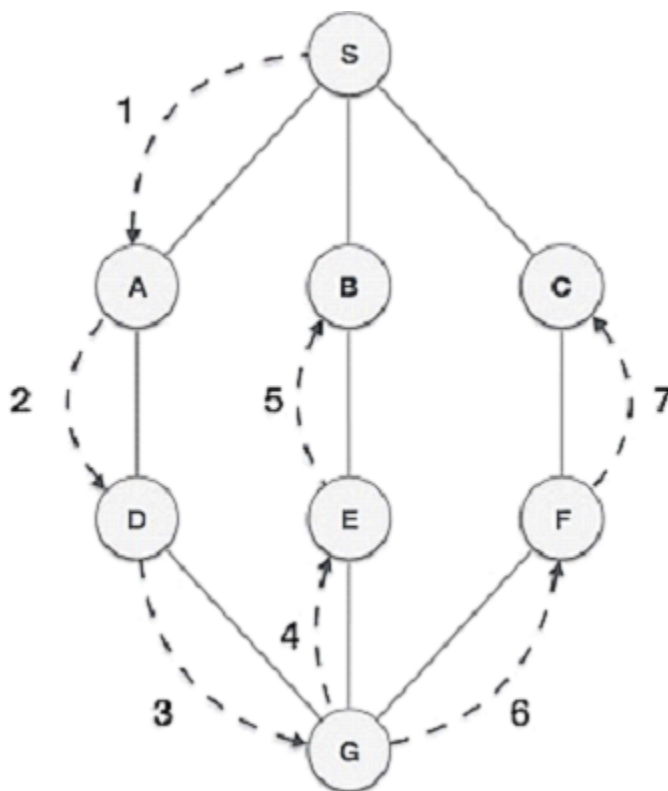


➤ Breadth-First Search Algorithm Solved Example Advantages and Disadvant...

Depth First Search: Algorithm traverses a graph in a depth-ward motion and uses a stack to remember to get the next vertex to start a search when a deadend occurs in any iteration.

DFS uses LIFO (Last In First Out) principle while using Stack to find the shortest path. DFS is also called Edge Based Traversal because it explores the nodes along the edge or path. DFS is faster and requires less memory. DFS is best suited for decision trees.

Example of DFS



- This strategy does not guarantee that the optimal solution has been found.
- In this strategy, search reaches a satisfactory solution more rapidly than breadth first, an advantage when the search space is large.

Algorithm :Depth-first search applies operators to each newly generated state, trying to drive directly toward the goal.

1. If the starting state is a goal state, quit and return success.

2. Otherwise, do the following until success or failure is signalled:
 - (a) Generate a successor E to the starting state. If there are no more successors, then signal failure.
 - (b) Call Depth-first Search with E as the starting state.
 - (c) If success is returned signal success; otherwise, continue in the loop.

Advantages 1. Low storage requirement: linear with tree depth.

2. Easily programmed: function call stack does most of the work of maintaining state of the search.

Disadvantages 1. May find a sub-optimal solution (one that is deeper or more costly than the best solution).

2. Incomplete: without a depth bound, may not find a solution even if one exists.

 [Depth First Search Algorithm Solved Example Advantages and Disadvantages by ...](#)

Heuristic Search: To find a solution in proper time rather than a complete solution in unlimited time we use heuristics. 'A heuristic function is a function that maps from problem state descriptions to measures of desirability, usually represented as numbers'.

Heuristic search methods use knowledge about the problem domain and choose promising operators first. These heuristic search methods use heuristic functions to evaluate the next state towards the goal state.

For finding a solution, by using the heuristic technique, one should carry out the following steps:

1. Add domain—specific information to select what is the best path to continue searching along.
2. Define a heuristic function $h(n)$ that estimates the 'goodness' of a node n . Specifically, $h(n)$ = estimated cost(or distance) of minimal cost path from n to a goal state.
3. The term heuristic means 'serving to aid discovery' and is an estimate, based on domain specific information that is computable from the current state description of how close we are to a goal.

For eg: Finding the route from one city to another city is an example of a search problem in which different search orders and the use of heuristic knowledge are easily understood.

1. State: The current city in which the traveller is located.
2. Operators: Roads linking the current city to other cities.
3. Cost Metric: The cost of taking a given road between cities.
4. Heuristic information: The search could be guided by the direction of the goal city from the current city, or we could use airline distance as an estimate of the distance to the goal.

Characteristics of heuristic search

- Heuristics are knowledge about a domain, which helps search and reasoning in its domain.
- Heuristic search incorporates domain knowledge to improve efficiency over blind search.
- Heuristic is a function that, when applied to a state, returns value as estimated merit of state, with respect to goal.
 - Heuristics might (for reasons) underestimate or overestimate the merit of a state with respect to goal.
 - Heuristics that underestimate are desirable and called admissible.
- Heuristic evaluation function estimates likelihood of given state leading to goal state.
- Heuristic search function estimates cost from current state to goal, presuming function is efficient.

Heuristic search is compared with Brute force or Blind search techniques

Blind Search	Heuristic Search
Can only search what it has knowledge about already.	Estimates 'distance' to goal state through explored nodes.
No knowledge about how far a node is from the goal state.	Guides search process toward goal. Prefers states (nodes) that lead close to and not away from goal state

Best First Search: is a traversal technique that decides which node is to be visited next by checking which node is the most promising one and then checking it. For this it uses an evaluation function to decide the traversal.

This best first search technique of tree traversal comes under the category of heuristic search or informed search technique.

The cost of nodes is stored in a priority queue. This makes implementation of best-first search the same as that of breadth First search. We will use the priorityqueue just like we use a queue for BFS.

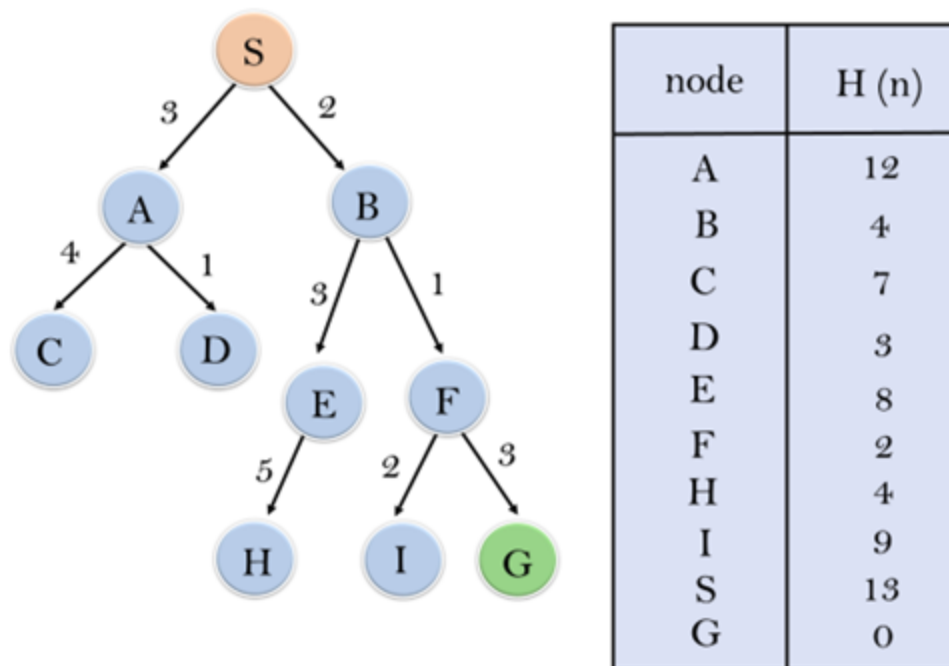
Unlike uninformed search where the agent blindly traverses to the next node in best first search a uses an evaluation function to determine which neighbor node would be most appropriate to move to. It uses the concept of a Priority queue and heuristic search.

Algorithm:

- Create two empty lists.
- Start from the initial node and add it to the ordered open list.
- Next the below steps are repeated until the final node or endpoint is reached.
- If the open list is empty exit the loop and return a False statement which says that the final node cannot be reached.
- Select the top node in the open list and move it to the closed list while keeping track of the parent node.
- If the node removed is the endpoint node, return a True statement meaning a path has been found and move the node to the closed list.
- However if it is not the endpoint node then list down all the neighboring nodes of it and add them to the open list.
- According to the evaluation function, reorder the nodes.

This algorithm will traverse the shortest path first in the queue. The time complexity of the algorithm is $O(n \cdot \log(n))$

Eg: Here we have a graph where our aim is to traverse from the node S to node G.

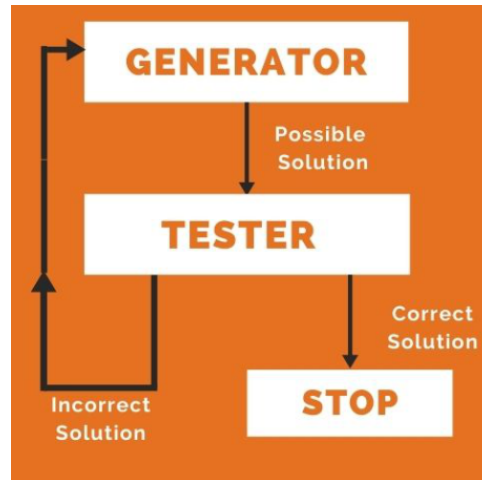


▶ Best First Search Algorithm with Solved Example in Artificial Intelligence ...

Generate and Test Method:

- Generate and Test Search is a heuristic search technique based on Depth First Search with Backtracking which guarantees to find a solution if done systematically and there exists a solution.
- In this technique, all the solutions are generated and tested for the best solution. It ensures that the best solution is checked against all possible generated solutions.
- It is also known as the British Museum Search Algorithm as it's like looking for an exhibit at random or finding an object in the British Museum by wandering randomly.
- The evaluation is carried out by the heuristic function as all the solutions are generated systematically in the generate and test algorithm but if there are some paths which are most unlikely to lead us to result then they are not considered.

- The heuristic does this by ranking all the alternatives and is often effective in doing so.
- Systematic Generate and Test may prove to be ineffective while solving complex problems.
- But there is a technique to improve in complex cases as well by combining generate and test search with other techniques so as to reduce the search space.



Algorithm:

1. Generate a possible solution. For example, generating a particular point in the problem space or generating a path for a start state.
2. Test to see if this is an actual solution by comparing the chosen point or the endpoint of the chosen path to the set of acceptable goal states
3. If a solution is found, quit. Otherwise go to Step 1

Eg: Dendral which infers the structure of organic compounds using NMR spectrogram also uses plan-generate-test.

📺 [Generate and Test Search Algorithm Solved Example in Artificial Intelligence by ...](#)

Hill Climbing:

- Hill climbing is a heuristic search algorithm that belongs to the family of local search methods.
- It is designed to solve problems where the goal is to find an optimal (or near-optimal) solution by iteratively moving from the current state to a better neighboring state, according to a heuristic or evaluation function.

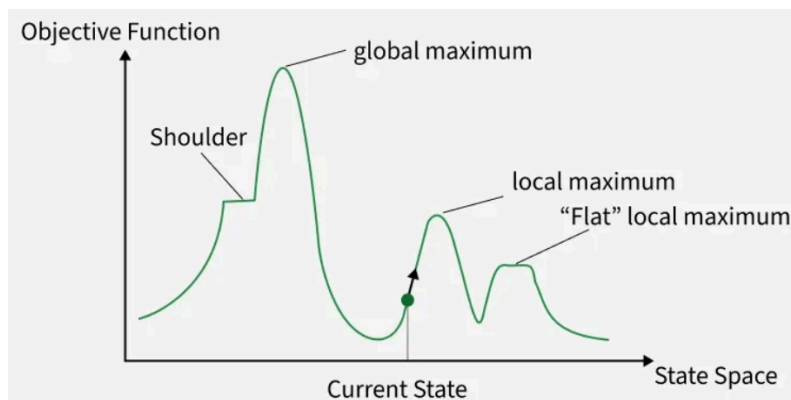
- It is an optimisation technique used in artificial intelligence (AI) to find solutions for a wide variety of problems.
- It operates on the principle of incrementally improving a solution by making local changes and evaluating their merit.
- Its simplicity, intuitive logic and adaptability to different problems make it a go-to method.

Algorithms

1. **Initial State:** Start with an arbitrary or random solution (initial state).
2. **Neighboring States:** Identify neighboring states of the current solution by making small adjustments (mutations or tweaks).
3. **Move to Neighbor:** If one of the neighboring states offers a better solution (according to some evaluation function), move to this new state.
4. **Termination:** Repeat this process until no neighboring state is better than the current one. At this point, we have reached a local maximum or minimum.

State-Space Diagram in Hill Climbing

The state-space diagram is a visual representation of all possible states the search algorithm can reach, plotted against the values of the objective function (the function we aim to maximize/ minimize).



In the state-space diagram:

- **X-axis:** Represents the state space which includes all the possible states or configurations that the algorithm can reach.
- **Y-axis:** Represents the values of the objective function corresponding to each state.

The optimal solution in the state-space diagram is represented by the state where the **objective function** reaches its maximum value, also known as the **global maximum**.

Key Regions in the State-Space Diagram

1. **Local Maximum:** A local maximum is a state better than its neighbors but not the best overall. While its objective function value is higher than nearby states, a global maximum may still exist.
2. **Global Maximum:** The global maximum is the best state in the state-space diagram where the objective function achieves its highest value. This is the optimal solution the algorithm seeks.
3. **Plateau/Flat Local Maximum:** A plateau is a flat region where neighboring states have the same objective function value, making it difficult for the algorithm to decide on the best direction to move.
4. **Ridge:** A ridge is a higher region with a slope which can look like a peak. This may cause the algorithm to stop prematurely, missing better solutions nearby.
5. **Current State:** The current state refers to the algorithm's position in the state-space diagram during its search for the optimal solution.
6. **Shoulder:** A shoulder is a plateau with an uphill edge allowing the algorithm to move toward better solutions if it continues searching beyond the plateau.

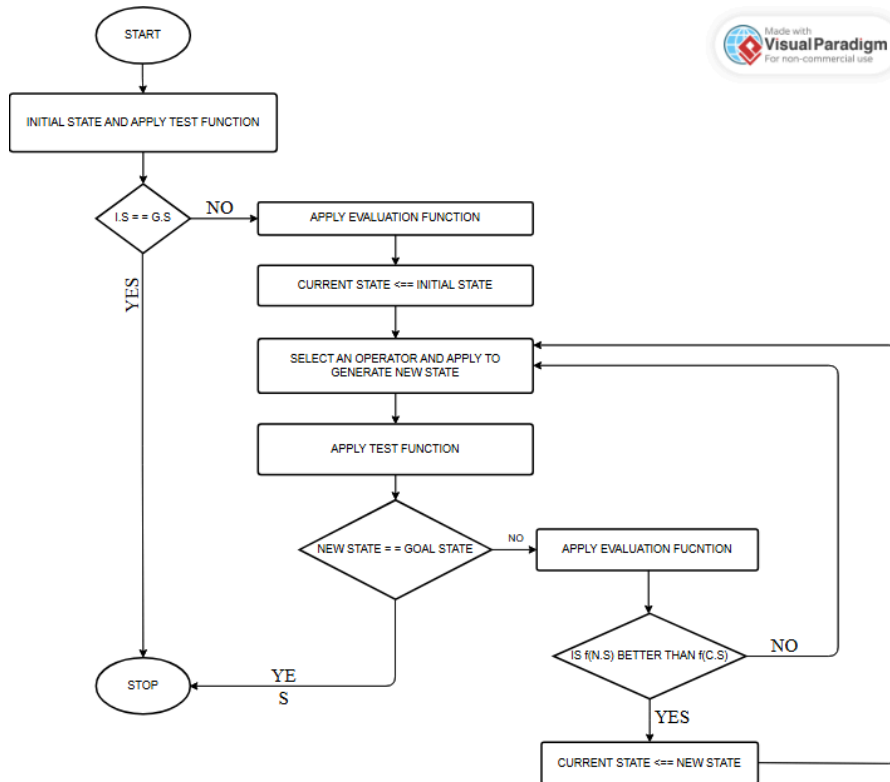
Types of Hill Climbing in Artificial Intelligence

1. Simple Hill Climbing Algorithm:

Simple hill climbing is the simplest way to implement a hill-climbing algorithm. It only evaluates the neighbor node state at a time and selects the first one that optimizes the current cost, and sets it as the current state. It only checks its one successor state, and if it finds a better state than the current state, then it moves, else it remains in the same state.

This algorithm has the following features:

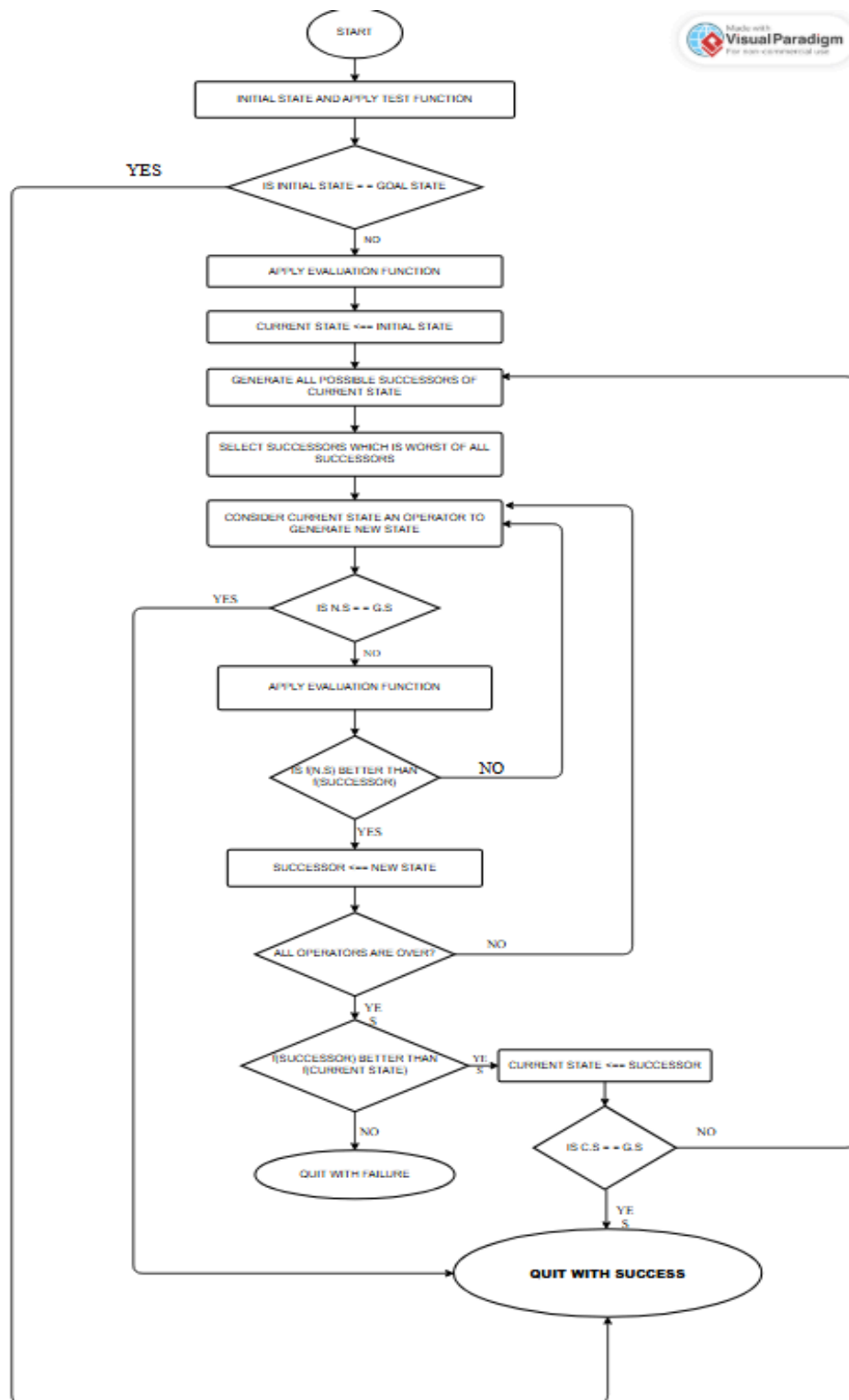
- Less time-consuming
- A less optimal solution, and the solution is not guaranteed



Algorithm for Simple Hill Climbing:

- 1: Evaluate the initial state. If it is the goal state, then return success and Stop.
- 2: Loop until a solution is found or there is no new operator left to apply.
- 3: Select and apply an operator to the current state.
- 4: Check the new state:
 - If it is the goal state, then return success and quit.
 - Else if it is better than the current state, then assign the new state as the current state.
 - Else if not better than the current state, then return to step 2.

2. Steepest-Ascent Hill Climbing: The steepest ascent algorithm is a variation of the simple hill climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node that is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors.



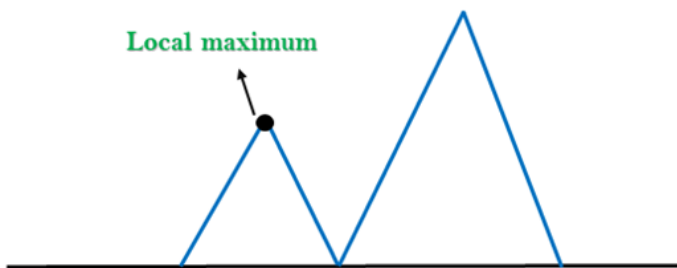
Algorithm for Steepest-Ascent hill climbing:

1. Evaluate the initial state. If it is the goal state, then return success and stop; else make the current state as the initial state.
2. Loop until a solution is found or the current state does not change.
 - a. Let SUCC be a state such that any successor of the current state will be better than it.
 - b. For each operator that applies to the current state:
 - Apply the new operator and generate a new state.
 - Evaluate the new state.
 - If it is the goal state, then return it and quit; else, compare it to the SUCC.
 - If it is better than SUCC, then set the new state as SUCC.
 - If the SUCC is better than the current state, then set the current state to SUCC.

3. Stochastic Hill Climbing: Stochastic hill climbing does not examine all its neighbors before moving. Rather, this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.

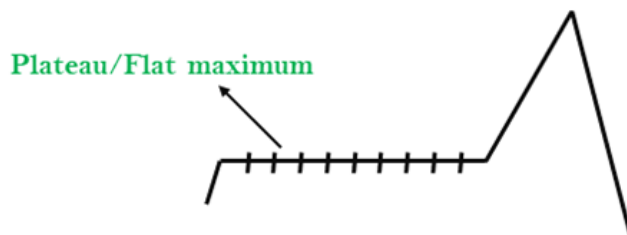
Problem and solutions(Hill climbing):

1. **Local Maximum:** A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.



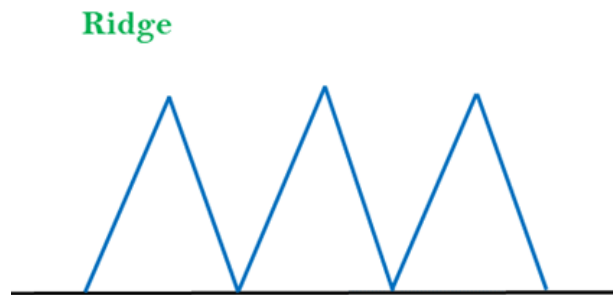
Solution: Backtracking technique can be a solution of the local maximum in state space landscape. Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.

2. **Plateau:** A plateau is the flat area of the search space in which all the neighboring states of the current state contain the same value, because this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.



Solution: The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find a non-plateau region.

3. **Ridges:** A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but it has a slope, and cannot be reached in a single move.



Solution: With the use of bidirectional search, or by moving in different directions, we can improve this problem.

A hill-climbing algorithm that never makes a move towards a lower value is guaranteed to be incomplete because it can get stuck on a local maximum. And if the algorithm applies a random walk, by moving a successor, then it may be complete but not efficient. **Simulated Annealing is an algorithm** which yields both efficiency and completeness.

(Annealing is a process of hardening a metal or glass to a high temperature, then cooling gradually, so this allows the metal to reach a low-energy crystalline state.)

The same process is used in simulated annealing, in which the algorithm picks a **random move, instead of picking the best move**. If the random move improves the state, then it follows the same path. Otherwise, the algorithm follows the path that has a **probability of less than 1, or it moves downhill and chooses another path**.

📺 Hill Climbing Algorithm Drawbacks Advantages Disadvantages Solved Example by Dr. ...

Applications of Hill climbing:

1. Machine Learning:
2. Robotics:
3. Network Design:
4. Game playing:
5. Natural language processing:

A* Algorithm:

- It is a search algorithm used to find the shortest path between an initial and a final point.
- It is often used for map traversal to find the shortest path.
- A* was initially designed as a graph traversal problem to help build a robot that can find its own course.
- It remains a widely popular algorithm for graph traversal.
- It searches for shorter paths first, thus making it an optimal and complete algorithm.
- An optimal algorithm will find the least cost outcome for a problem, while a complete algorithm finds all the possible outcomes of a problem.
- Another aspect that makes A* so powerful is its implementation of weighted graphs.
- A weighted graph uses numbers to represent the cost of taking each path or course of action. This means that the algorithms can take the path with the least cost, and find the best route in terms of distance and time.
- A major **drawback of the algorithm** is its space and time complexity. It takes a large amount of space to store all possible paths and a lot of time to find them.
- A* (A-Star) is a popular choice for pathfinding because it combines the strengths of **Dijkstra's algorithm and Greedy Best-First Search**, making it efficient and optimal.
- It uses a heuristic function to evaluate paths by balancing the cost to reach a node (g-cost) and the estimated cost to the goal (h-cost).
- It is an extension of Dijkstra's shortest path algorithm (Dijkstra's Algorithm).
- The extension here is that, instead of using a priority queue to store all the elements, we use heaps (binary trees) to store them.
- The A* Search Algorithm also uses a heuristic function that provides additional information regarding how far away from the goal node we are.

- This function is used in conjunction with the f-heap data structure in order to make searching more efficient.
- At each step, it picks the node with the smallest value of 'f' (the sum of 'g' and 'h') and processes that node/cell. 'g' and 'h' is defined as simply as possible below:
 - 'g' is the distance it takes to get to a certain square on the grid from the starting point, following the path we generated to get there.
 - 'h' is the heuristic, which is the estimation of the distance it takes to get to the finish line from that square on the grid.

Algorithm:

- ***g(n): The actual cost path from the start state to the current state.***
- ***h(n): The actual cost path from the current state to the goal state.***
- ***f(n): The actual cost path from the start state to the goal state.***
- For the implementation of the A* algorithm, we will use two arrays namely OPEN and CLOSE.
- **OPEN:** An array that contains the nodes that have been generated but have not been yet examined.
- **CLOSE:** An array that contains the nodes that have been examined.

1: Place the starting node into OPEN and find its f (n) value.

2: Remove the node from OPEN, having the smallest f (n) value. If it is a goal node then stop and return success.

3: Else remove the node from OPEN, find all its successors.

4: Find the f(n) value of all successors; place them into OPEN and place the removed node into CLOSE.

5: Go to Step-2.

6: Exit.

Heuristics(value calculation)

We can easily calculate g, but **how do we calculate h?**

There are two methods that we can use to calculate the value of h:

1. Determine h's exact value (which is certainly time-consuming).

(or)

2. Utilize various techniques to approximate the value of h. (less time-consuming).
Let us discuss both methods.

Exact Heuristics

Although we can obtain exact values of h, doing so usually takes a very long time.
The ways to determine h's precise value are listed below.

1. Before using the A* Search Algorithm, pre-calculate the distance between every pair of cells.
2. Using the distance formula/Euclidean Distance, we may directly determine the precise value of h in the absence of blocked cells or obstructions.

Approximation Heuristics(calculate Approximation Heuristics.)

To determine h, there are typically three approximation heuristics:

1. Manhattan Distance

The Manhattan Distance is the total of the absolute values of the discrepancies between the x and y coordinates of the current and the goal cells.

The formula is summarized below -

$$h = \text{abs}(\text{curr_cell.x} - \text{goal.x}) + \text{abs}(\text{curr_cell.y} - \text{goal.y})$$

We must use this heuristic method when we are only permitted to move in four directions - top, left, right, and bottom.

Let us now take a look at the Diagonal Distance method to calculate the heuristic.

2. Diagonal Distance

It is nothing more than the greatest absolute value of differences between the x and y coordinates of the current cell and the goal cell.

This is summarized below in the following formula -

$$dx = \text{abs}(\text{curr_cell.x} - \text{goal.x})$$
$$dy = \text{abs}(\text{curr_cell.y} - \text{goal.y})$$
$$h = D * (dx + dy) + (D2 - 2 * D) * \min(dx, dy)$$

where D is the length of every node (default = 1) and D2 is the diagonal

We use this heuristic method when we are permitted to move only in eight directions, like the King's moves in Chess.

Let us now take a look at the Euclidean Distance method to calculate the heuristic.

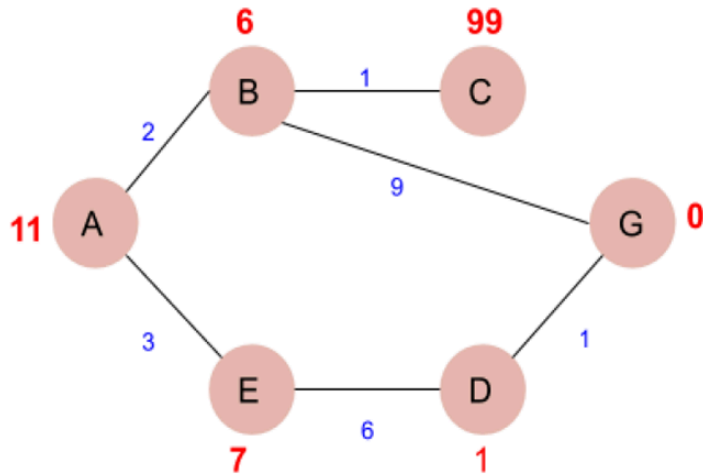
3. Euclidean Distance

The Euclidean Distance is the distance between the goal cell and the current cell using the distance formula:

$$h = \text{sqrt} \left((\text{curr_cell.x} - \text{goal.x})^2 + (\text{curr_cell.y} - \text{goal.y})^2 \right)$$

We use this heuristic method when we are permitted to move in any direction of our choice..

Example:



Numbers written on edges represent the distance between nodes. Numbers written on nodes represent the heuristic value.

Given the graph, find the cost-effective path from A to G. That is A is the source node and G is the goal node.

Now from A, we can go to point B or E, so we compute $f(x)$ for each of them,

$$A \rightarrow B = g(B) + h(B) = 2 + 6 = 8$$

$$A \rightarrow E = g(E) + h(E) = 3 + 7 = 10$$

Since the cost for $A \rightarrow B$ is less, we move forward with this path and compute the $f(x)$ for the children nodes of B.

Now from B, we can go to point C or G, so we compute $f(x)$ for each of them,

$$A \rightarrow B \rightarrow C = (2 + 1) + 99 = 102$$

$$A \rightarrow B \rightarrow G = (2 + 9) + 0 = 11$$

Here the path $A \rightarrow B \rightarrow G$ has the least cost but it is still more than the cost of $A \rightarrow E$, thus we explore this path further.

Now from E, we can go to point D, so we compute $f(x)$,

$$A \rightarrow E \rightarrow D = (3 + 6) + 1 = 10$$

Comparing the cost of $A \rightarrow E \rightarrow D$ with all the paths we got so far and as this cost is least of all we move forward with this path.

Now compute the $f(x)$ for the children of D

$$A \rightarrow E \rightarrow D \rightarrow G = (3 + 6 + 1) + 0 = 10$$

Now comparing all the paths that lead us to the goal, we conclude that **$A \rightarrow E \rightarrow D \rightarrow G$** is the most cost-effective path to get from A to G.

Advantages of A* Star

- It is complete and optimal.
- It is the best one from other techniques.
- It is used to solve very complex problems.
- It is optimally efficient, i.e. there is no other optimal algorithm guaranteed to expand fewer nodes than A*.

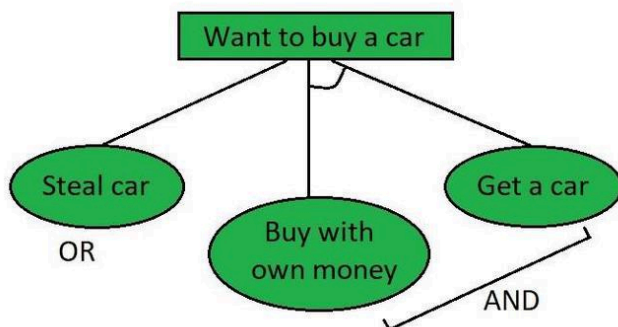
Disadvantages of A* Star

- This algorithm is complete if the branching factor is finite and every action has a fixed cost.
- The speed execution of the A* search is highly dependent on the accuracy of the heuristic algorithm that is used to compute $h(n)$.
- It has complexity problems.

📺 1. A star Search Algorithm to Move from start state to final state 8 Puzzle Problem ...

AO* Algorithm

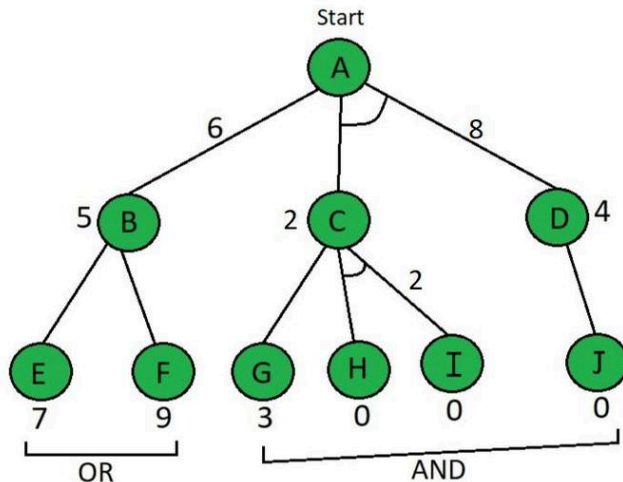
- It is an extension of the A* algorithm, designed to handle more complex problems that require handling multiple paths and making decisions at each node.
- The **AO algorithm*** (short for "**And-Or Star**") is a powerful **best-first search** method used to solve problems that can be represented as a **directed acyclic graph (DAG)**.
- Unlike traditional algorithms like A*, which explore a single path, the AO* algorithm evaluates multiple paths simultaneously. This makes it more efficient for problems that involve **AND-OR nodes**.
- The AND nodes represent tasks where all child nodes must be satisfied, while OR nodes offer multiple alternative paths where only one child node needs to be satisfied to achieve the goal.
- **Example: We consider the process of buying a car.**
- This process can be broken down into smaller tasks using an **AND-OR graph**.
- The AND section might include securing funds or applying for a loan.
- The OR section could present different options for acquiring the car, such as purchasing with cash or financing.
- Each subproblem in the AND-OR graph must be solved before the main issue is resolved.



- By leveraging heuristics, it identifies the best path, significantly reducing time complexity.

- AO* is more efficient when solving problems represented as **AND-OR graphs**, as it evaluates multiple paths at once.

Lets understand using an example of graph:



- Each node in the graph is assigned a heuristic value, denoted as **$h(n)$** , and the edge length is considered as **1**.

1: Initial Evaluation Using $f(n) = g(n) + h(n)$

2: Explore Nodes with low value.

3: Compare and Explore Paths

Step 1. Starting from node **A**, we use the evaluation function:

$$\begin{aligned}
 f(A \rightarrow B) &= g(B) + h(B) \\
 &= 1 + 5 \quad (g(n) = 1 \text{ is the default path cost}) \\
 &= 6
 \end{aligned}$$

For the path involving **AND** nodes (**C** and **D**):

$$\begin{aligned}
 f(A \rightarrow C + D) &= g(C) + h(C) + g(D) + h(D) \\
 &= 1 + 2 + 1 + 4 \quad (C \text{ \& } D \text{ are AND nodes}) \\
 &= 8
 \end{aligned}$$

Since $f(A \rightarrow B) = 6$ is smaller than $f(A \rightarrow C + D) = 8$, we select the path $A \rightarrow B$.

Step 2: Next, we explore node **B**, and calculate the values for nodes **E** and **F**:

$$\begin{aligned}
 f(B \rightarrow E) &= g(E) + h(E) \\
 &= 1 + 7 \\
 &= 8
 \end{aligned}$$

$$\begin{aligned}
 f(B \rightarrow F) &= g(F) + h(F) \\
 &= 1 + 9 \\
 &= 10
 \end{aligned}$$

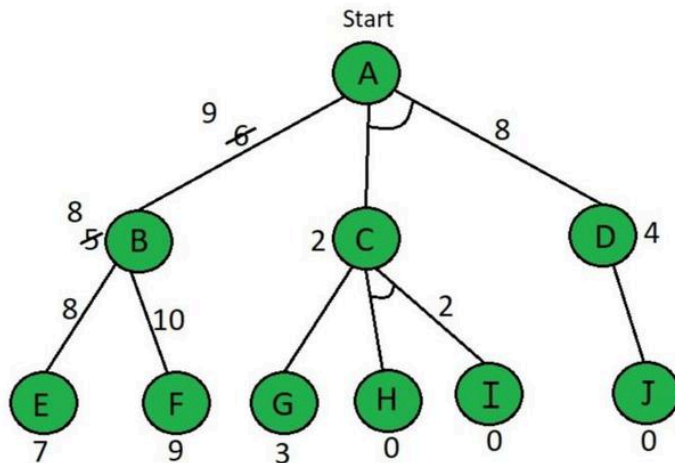
B→E path is chosen which is minimum path

B's heuristic value is different from its actual value The heuristic is updated and the minimum cost path is selected. The minimum value in our situation is 8.

The heuristic for A must be updated due to the change in B's heuristic.

*Thus, $f(B \rightarrow E) = 8$ is chosen as the optimal path. Since **B**'s heuristic value differs from its actual cost, we update the heuristic for **A**.*

$$\begin{aligned}
 f(A \rightarrow B) &= g(B) + \text{updated } h(B) \\
 &= 1 + 8 \\
 &= 9
 \end{aligned}$$



Step3: Now, we compare $f(A \rightarrow B) = 9$ with $f(A \rightarrow C + D) = 8$. Since $f(A \rightarrow C + D)$ is smaller, we explore this path and move to node **C**.

For node **C**:

$$\begin{aligned} f(C \rightarrow G) &= g(G) + h(G) \\ &= 1 + 3 \\ &= 4 \end{aligned}$$

$$\begin{aligned} f(C \rightarrow H + I) &= g(H) + h(H) + g(I) + h(I) \\ &= 1 + 0 + 1 + 0 \quad (H \text{ \& } I \text{ are AND nodes}) \\ &= 2 \end{aligned}$$

$f(C \rightarrow H+I)$ is selected as the path with the lowest cost and the heuristic is also left unchanged because it matches the actual cost.

Paths H & I are solved because the heuristic for those paths is **0**, but Path $A \rightarrow D$ needs to be calculated because it has an **AND**.

The path $f(C \rightarrow H + I) = 2$ is selected. Since the heuristic for **H** and **I** matches the actual cost (both are 0), these paths are considered solved. Next, we calculate the value for $A \rightarrow D$ as it also has an **AND** node.

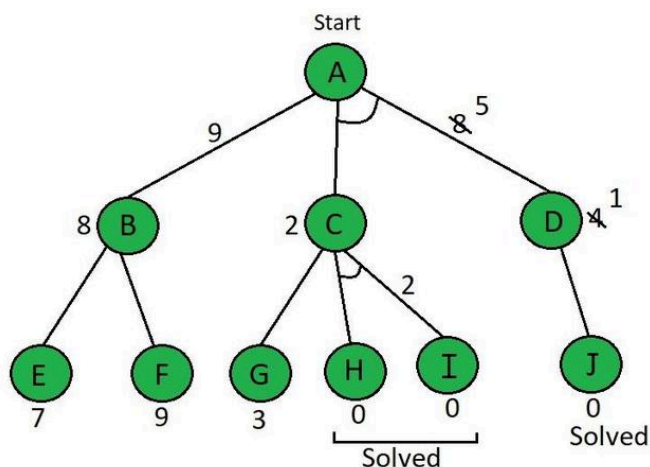
For node **D**:

$$\begin{aligned}
 f(D \rightarrow J) &= g(J) + h(J) \\
 &= 1 + 0 \\
 &= 1
 \end{aligned}$$

After updating the heuristic for **D**, we recalculate:

$$\begin{aligned}
 f(A \rightarrow C + D) &= g(C) + h(C) + g(D) + h(D) \\
 &= 1 + 2 + 1 + 1 \\
 &= 5
 \end{aligned}$$

Now that **f(A -> C + D)** has the lowest cost, this becomes the solved path, and the **AND-OR graph** is now fully resolved.



workflow of the *AO algorithm** involves first calculating heuristic values for each level, then updating these values from the leaf nodes upward to the root node.

Advantages of AO* Star

- It is an optimal algorithm.
- If traverse according to the ordering of nodes.
- It can be used for both OR and AND graphs.

Disadvantages of AO* Star

- Sometimes for unsolvable nodes, it can't find the optimal path.
- Its complexity is better than other algorithms.

Iterative Deepening A* :

- *Iterative deepening A (IDA)** is a powerful graph traversal and pathfinding algorithm designed to find the **shortest path in a weighted graph**.
- This method combines features of **iterative deepening depth-first search (IDDFS)** and the *A search algorithm** by using a heuristic function to estimate the remaining cost to the goal node.
- IDA* is often referred to as a **memory-efficient version of A***, as it requires significantly less memory while still ensuring optimal pathfinding.
- Working: The IDA* algorithm works by incrementally increasing the **threshold** based on the f-score of each node, which is calculated using the formula:

$$f(n)=g(n)+h(n)$$

$$f(n)=\text{Actualcost}+\text{Estimated cost}$$

Where **h** is admissible.

Here,

- $f(n)$ = Total cost evaluation function.
- $g(n)$ = The actual cost from the initial node to the current node.
- $h(n)$ = Heuristic estimated cost from the current node to the goal state.
- It is based on the approximation according to the problem characteristics.

- F-score is a **heuristic function** that is used to estimate the cost of reaching the goal state from a given state. It is a combination of two other heuristic functions, **$g(n)$** and **$h(n)$** .
- **A lower F-score indicates that a node is closer to the goal state and will be expanded before a node with a higher F-score.**

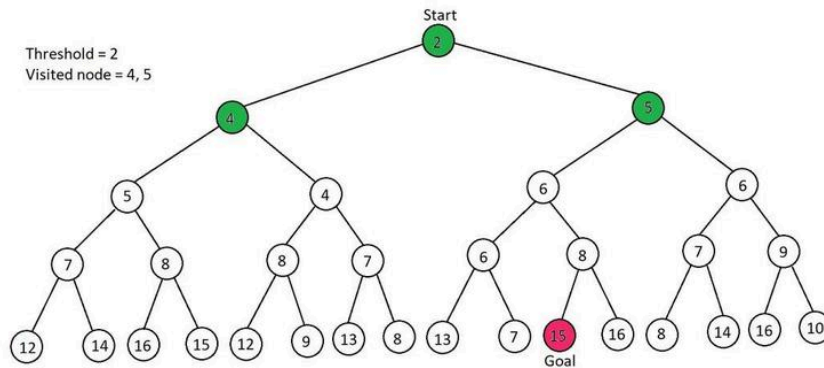
Algorithm:

1. **Initialization:** Set the root node as the current node and compute its f-score.
2. **Set Threshold:** Initialize a threshold based on the f-score of the starting node.
3. **Node Expansion:** Expand the current node's children and calculate their f-scores.
4. **Pruning:** If the f-score exceeds the threshold, prune the node and store it for future exploration.
5. **Path Return:** Once the goal node is found, return the path from the start node to the goal.
6. **Update Threshold:** If the goal is not found, increase the threshold based on the minimum pruned value and repeat the process.

Example: Understand using the graph

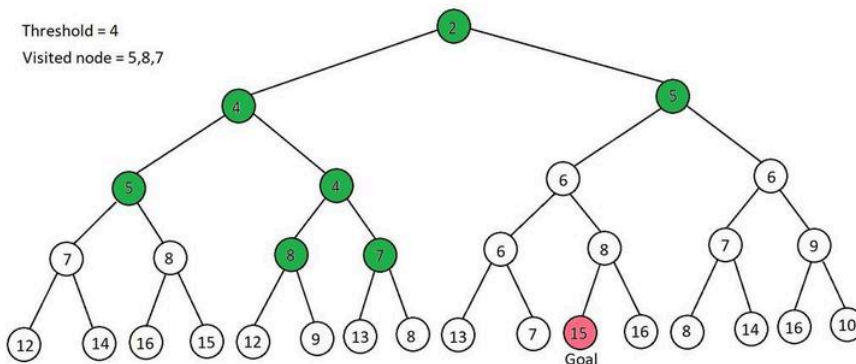
- The **f score** is written inside the nodes means the f score is already computed and the start node is **2** whereas the goal node is **15**.
- The explored node is colored green.
- Need to reach goal using IDA*.

ITERATION 1



- Root node as current node i.e 2
- Threshold = current node value ($2=2$). So explore its children.
- $4 > \text{Threshold}$ & $5 > \text{Threshold}$. So, this iteration is over and the pruned values are 4, and 5.

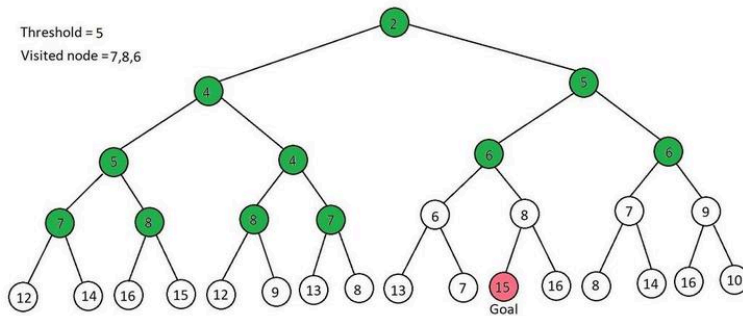
ITERATION 2



- In pruned values, the least is 4, So threshold = 4
- current node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, first children 4, So, set current node = 4 i.e equal to the threshold, so, explored its children also i.e 5, 4 having $5 > \text{threshold}$ so, pruned it and explore second child of node 4 i.e 4, so set current node = 4 = threshold, and explore its children i.e 8 & 7 having both 8 & 7 $> \text{threshold}$ so, pruned it. At the end of this, our pruned value is 5, 8, 7

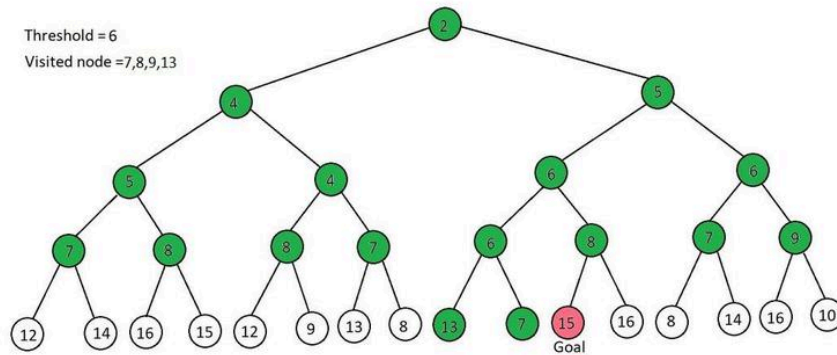
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 > \text{threshold}$, So pruned it.
- So, our pruned value is 5,8,7.

ITERATION 3



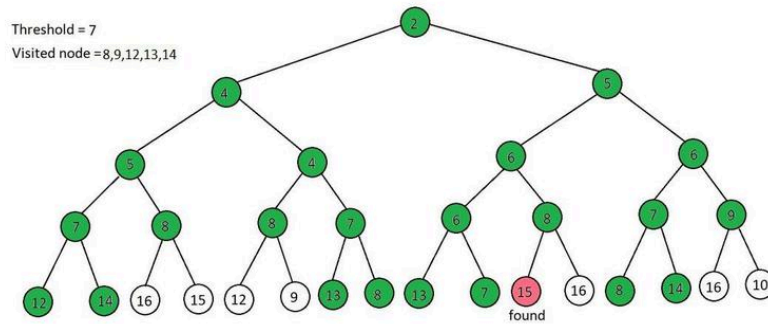
- In pruned values, the least is 5, So threshold = 5
- current node = root node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, first children 4, So, set current node = $4 < \text{threshold}$, so, explore its children also i.e 5, 4 having $5 = \text{threshold}$ so explore its child also 7 & $8 > \text{threshold}$. So, pruned it and explored the second child of node 4 i.e 6, so set current node = $6 < \text{threshold}$, and explore its children i.e 7 & 8 here, both $7 & 8 > \text{threshold}$ so, pruned it. At the end of this, our pruned value is 7 & 8
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 = \text{threshold}$, so, explore its children also i.e 6 & 6, i.e both $6 & 6 > \text{threshold}$. So pruned it
- So, our pruned value is 7,8 & 6

ITERATION 4:



- In pruned values, the least value is 6, So threshold = 6
- current node = root node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, the first child is 4, So, set current node = $4 < \text{threshold}$, so, explored its children also i.e 5, 4 having $5 < \text{threshold}$ so explore its child also 7 & 8 $> \text{threshold}$. So, pruned it and explore the second child of node 4 i.e 4, so set current node = $4 < \text{threshold}$, and explore its children i.e 8 & 7 here, both 8 & 7 $> \text{threshold}$ so, pruned it. At the end of this, our pruned value is 7 & 8
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 = \text{threshold}$, so, explored its children also i.e 6 & 6, i.e both 6 & 6 = threshold, So, explore one by one,
- The first 6 has two children i.e 6 & 8, having $6 = \text{threshold}$. So, explore its child also i.e 13 & 7. here both 13 & 7 $> \text{Threshold}$. So, pruned it. next is 8 $> \text{Threshold}$. pruned it, So, pruned value at this stage is 13,7 & 8.
- Explore the second child of 5 i.e 6 = Threshold. So, explore its child i.e 7 & 9. Both are greater than Threshold. So, pruned it
- So, our pruned values are 13,7,8 & 9.

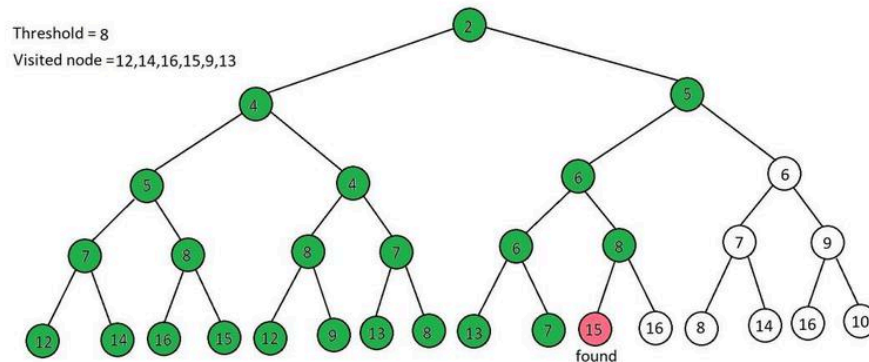
ITERATION 5:



- In pruned values, the least value is 7, So threshold = 7
- current node = root node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, the first child is 4, So, set current node = $4 < \text{threshold}$, so, explored its children also i.e 5, 4
- The first child of 4 is 5 i.e $5 < \text{threshold}$ so explore its child also 7&8, Here $7 = \text{threshold}$. So, explore its children i.e 12 & 14, both $> \text{Threshold}$. So, pruned it. And the second child of 5 is $8 > \text{Threshold}$, So, pruned it. At this stage, our pruned value is 12, 14 & 7.
- Now explore the second child of node 4 i.e 4, so set current node = $4 < \text{threshold}$, and explore its children i.e 8 & 7 here, $8 > \text{threshold}$ so, pruned it. then go to the second child i.e $7 = \text{Threshold}$, So explore its children i.e 13 & 8. having both $> \text{Threshold}$. So pruned it. At the end of this, our pruned value is 12,14,8 & 13
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 < \text{threshold}$, so, explored its children also i.e 6 & 6, i.e both $6 \& 6 < \text{threshold}$, So, explore one by one,
- The first 6 has two children i.e 6 & 8, having $6 < \text{threshold}$. So, explore its child also i.e 13 & 7. here $13 > \text{Threshold}$. So, pruned it. And $7 = \text{Threshold}$. And it hasn't any child. So, the shift to the next sub-child of 6 i.e $8 > \text{threshold}$, So, pruned it. The pruned value at this stage is 12,14,8 & 13
- Explore the second child of 5 i.e $6 < \text{Threshold}$. So, explore its child i.e 7 & 9. Here $7 = \text{Threshold}$, So, explore its children i.e 8 & 14, Both are greater than Threshold. So, pruned it, Now the sub child of 6 is $9 > \text{Threshold}$, So, pruned it.

- So, our pruned values are 12,14,8,13 & 9.

ITERATION 6:



- In pruned values, the least value is 8, So threshold = 8
- current node = root node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, the first child is 4, So, set current node = $4 < \text{threshold}$, so, explored its children also i.e 5, 4
- The first child of 4 is 5 i.e $5 < \text{threshold}$ so explore its child also 7&8, Here $7 < \text{threshold}$. So, explore its children i.e 12 & 14, both $> \text{Threshold}$. So, pruned it. And the second child of 5 is $8 = \text{Threshold}$, So, So, explore its children i.e 16 & 15, both $> \text{Threshold}$. So, pruned it. At this stage, our pruned value is 12, 14, 16 & 15.
- Now explore the second child of node 4 i.e 4, so set current node = $4 < \text{threshold}$, and explore its children i.e 8 & 7 here, $8 = \text{Threshold}$, So, So, explore its children i.e 12 & 9, both $> \text{Threshold}$. So, pruned it. then go to the second child i.e $7 < \text{Threshold}$, So explore its children i.e 13 & 8. having $13 > \text{Threshold}$. So pruned it. and $8 = \text{Threshold}$ and it hasn't any child. At the end of this, our pruned values are 12, 14, 16, 15, and 13.
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 < \text{threshold}$, so, explored its children also i.e 6 & 6, i.e both $6 \& 6 < \text{threshold}$, So, explore one by one,

- The first 6 has two children i.e 6 & 8, having $6 < \text{threshold}$. So, explore its child also i.e 13 & 7. here $13 > \text{Threshold}$. So, pruned it. And $7 < \text{Threshold}$. And it doesn't have any child. So, the shift to the next sub-child of 6 i.e 8 = threshold. So, explored its children also i.e 15 & 16, Here **15 = Goal Node**. So, stop this iteration. Now no need to explore more.
- The goal path is **2-->5-->6-->8-->15**

Real world Application of IDA*: solving n- Queen problem, sliding puzzle game.

Advantages

1. **Optimal Pathfinding:** IDA* guarantees finding the optimal path, as it never overestimates the cost to the goal.
2. **Memory Efficient:** It uses limited memory compared to A* by applying depth-first search techniques.
3. **Admissible Heuristic:** The heuristic function ensures that the algorithm remains admissible, thus optimizing performance.
4. **Efficient with Large State Spaces:** IDA* handles large graphs efficiently by pruning unnecessary nodes.

Disadvantages

1. **Repeated Node Exploration:** The algorithm does not store visited nodes, leading to repeated exploration.
 2. **Slower than A*:** IDA* can be slower than algorithms like A* due to the repeated exploration of nodes.
 3. **Higher Computational Cost:** It may take longer and consume more processing power compared to other algorithms like A* or breadth-first search.
-

Game Playing->Overview:

- GamePlaying is an important domain of Artificial Intelligence.
- There are two reasons that games appeared to be a good domain.
 1. They provide a structured task in which it is very easy to measure success or failure.
 2. They are easily solvable by a straightforward search from the starting state to a winning position.
- Games require only domain knowledge such as the rules, legal moves and the conditions of winning or losing the game.
- In a two-player game, both the players try to win the game. So, both of them try to make the best move possible at each turn.
- To improve the effectiveness of a search based problem solving program two things can be done.
 1. Improve generate procedure so that only good moves are generated.
 2. Improve the test procedure so that the best move will be recognized and explored first.
- If we use a legal-move generator then the test procedure will have to look at each of them, because the test procedure must look at so many possibilities and it must be fast.
- The depth of the resulting tree or graph and its branching factor will be too large.
- Instead of a legal-move generator we can use plausible-move generator in which only some small numbers of promising moves are generated.
- As the number of legal available moves increases it becomes increasingly important in applying heuristics to select only those moves that seem more promising.
- The performance of the overall system can be improved by adding heuristic knowledge into both the generator and the tester.
- It is possible to search a tree only ten or twenty moves deep then in order to choose the best move, the resulting board positions must be compared to discover which is most advantageous.
- This is done using a static evaluation function, which uses whatever information it has to evaluate Individual board position by estimating how likely they are to lead eventually to a win.

Applications:

Expert System:

Introduction:

- Expert Systems are a branch of artificial intelligence (AI) that simulates the decision-making abilities of a human expert in solving complex problems within a specific domain.
- The goal of an expert system is to mimic the expertise of a human specialist in a particular field, such as those gained through Data Science Training making it accessible to individuals who may not have the same level of expertise.

- These systems are designed to solve problems by reasoning through bodies of knowledge, represented mainly as rules within the system.
- They are widely used in various sectors, such as medicine, engineering, finance, and more, to provide assistance, recommendations, and solutions to problems.
- An Expert System typically consists of two main components: a **knowledge base** and an **inference engine**.
- These systems work by applying logical reasoning to a set of facts, ultimately deriving conclusions and solutions that are as accurate as possible.
- Expert Systems are seen as valuable tools for decision-making processes and have proven essential for industries where specialized knowledge is crucial, yet the availability of human experts is limited.

Characteristics of Expert System:

- High performance
- Understandable
- Reliable
- Highly responsive

Capabilities of Expert Systems

The expert systems are capable of

- Advising
- Instructing and assisting human in decision making
- Demonstrating
- Deriving a solution
- Diagnosing
- Explaining
- Interpreting input
- Predicting results
- Justifying the conclusion
- Suggesting alternative options to a problem

They are incapable of

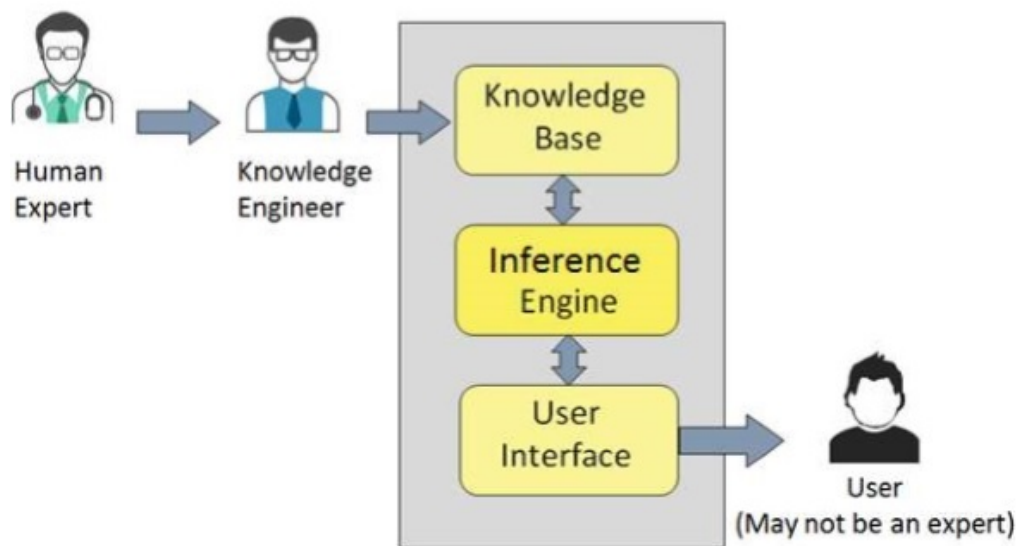
- Substituting human decision makers
- Possessing human capabilities
- Producing accurate output for inadequate knowledge base
- Refining their own knowledge

Components of Expert Systems

The components of ES include:

- Knowledge Base
- Inference Engine
- User Interface

Let us see them one by one briefly



Knowledge Base

It contains domain-specific and high-quality knowledge.

Knowledge is required to exhibit intelligence. The success of any ES majorly depends upon the collection of highly accurate and precise knowledge.

What is Knowledge?

The data is collection of facts. The information is organized as data and facts about the task domain. **Data**, **information**, and **past experience** combined together are termed as knowledge.

Components of Knowledge Base

The knowledge base of an ES is a store of both, factual and heuristic knowledge.

Factual Knowledge: It is the information widely accepted by the Knowledge Engineers and scholars in the task domain.

Heuristic Knowledge: It is about practice, accurate judgment, ones ability of evaluation, and guessing.

Knowledge representation

It is the method used to organize and formalize the knowledge in the knowledge base. It is in the form of IF-THEN-ELSE rules.

Knowledge Acquisition

The success of any expert system majorly depends on the quality, completeness, and accuracy of the information stored in the knowledge base.

The knowledge base is formed by readings from various experts, scholars, and the **Knowledge Engineers**. The knowledge engineer is a person with the qualities of empathy, quick learning, and case analyzing skills.

He acquires information from subject experts by recording, interviewing, and observing him at work, etc. He then categorizes and organizes the information in a meaningful way, in the form of IF-THEN-ELSE rules, to be used by an inference machine. The knowledge engineer also monitors the development of the ES.

Inference Engine

Use of efficient procedures and rules by the Inference Engine is essential in deducting a correct, flawless solution.

In case of knowledge-based ES, the Inference Engine acquires and manipulates the knowledge from the knowledge base to arrive at a particular solution.

In case of rule based ES, it –

- Applies rules repeatedly to the facts, which are obtained from earlier rule application.

- Adds new knowledge into the knowledge base if required.

Resolves rules conflict when multiple rules are applicable to a particular case.

To recommend a solution, the Inference Engine uses the following strategies –

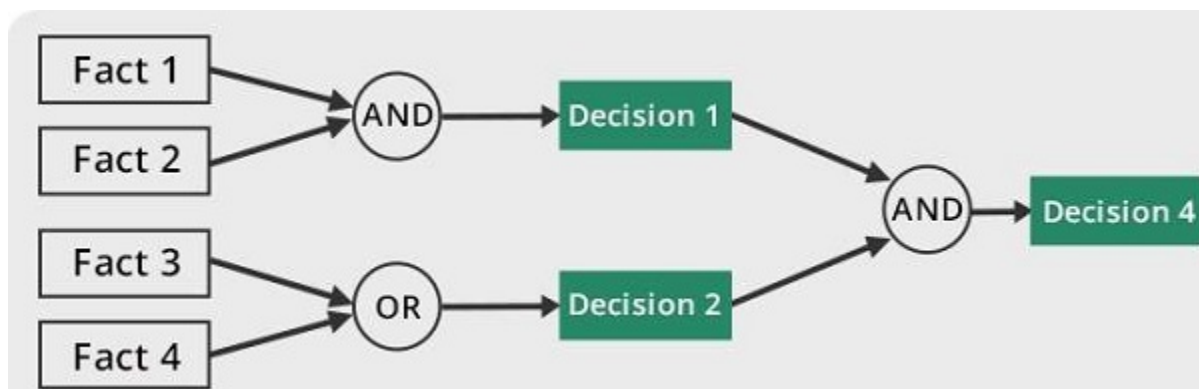
Forward Chaining
Backward Chaining

Forward Chaining

It is a strategy of an expert system to answer the question, **What can happen next?**

Here, the Inference Engine follows the chain of conditions and derivations and finally deduces the outcome. It considers all the facts and rules, and sorts them before concluding to a solution.

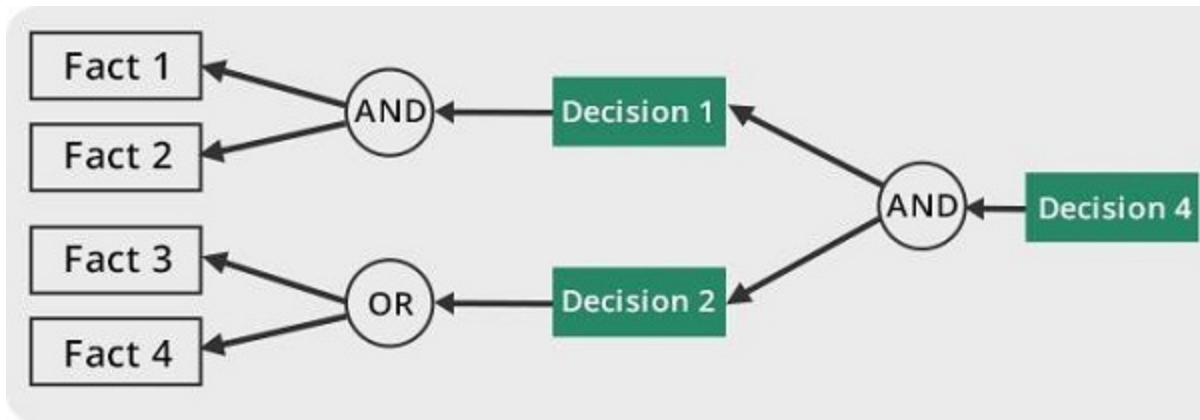
This strategy is followed for working on conclusion, result, or effect. For example, prediction of share market status as an effect of changes in interest rates.



Backward Chaining

With this strategy, an expert system finds out the answer to the question, **Why this happened?**

On the basis of what has already happened, the Inference Engine tries to find out which conditions could have happened in the past for this result. This strategy is followed for finding out cause or reason. For example, diagnosis of blood cancer in humans.



User Interface

User interface provides interaction between user of the ES and the ES itself. It is generally Natural Language Processing so as to be used by the user who is well-versed in the task domain. The user of the ES need not be necessarily an expert in Artificial Intelligence.

It explains how the ES has arrived at a particular recommendation. The explanation may appear in the following forms –

- Natural language displayed on screen.
- Verbal narrations in natural language.
- Listing of rule numbers displayed on the screen.

The user interface makes it easy to trace the credibility of the deductions.

Requirements of Efficient ES User Interface

- It should help users to accomplish their goals in shortest possible way.
- It should be designed to work for users existing or desired work practices.
- Its technology should be adaptable to users requirements; not the other way round.
- It should make efficient use of user input.

Expert Systems Limitations

No technology can offer easy and complete solution. Large systems are costly, require significant development time, and computer resources. ESs have their limitations which include –

- Limitations of the technology
- Difficult knowledge acquisition
- ES are difficult to maintain
- High development costs

Applications of Expert System

The following table shows where ES can be applied.

Application	Description
Design Domain	Camera lens design, automobile design.
Medical Domain	Diagnosis Systems to deduce cause of disease from observed data, conduction medical operations on humans.
Monitoring Systems	Comparing data continuously with observed system or with prescribed behavior such as leakage monitoring in long petroleum pipeline.
Process Control Systems	Controlling a physical process based on monitoring.
Knowledge Domain	Finding out faults in vehicles, computers.
Finance/Commerce	Detection of possible fraud, suspicious transactions, stock market trading, Airline scheduling, cargo scheduling.

Expert System Technology

There are several levels of ES technologies available. Expert systems technologies include –

Expert System Development Environment

The ES development environment includes hardware and tools. They are –

Workstations, minicomputers, mainframes.

High level Symbolic Programming Languages such as **LIS**t **P**rogramming (LISP) and **P**ROgrammation en **L**OGique (PROLOG).
Large databases.

Tools

They reduce the effort and cost involved in developing an expert system to large extent.

Powerful editors and debugging tools with multi-windows.

They provide rapid prototyping

Have Inbuilt definitions of model, knowledge representation, and inference design.

Shells

A shell is nothing but an expert system without knowledge base. A shell provides the developers with knowledge acquisition, inference engine, user interface, and explanation facility. For example, few shells are given below –

Java Expert System Shell (JESS) that provides fully developed Java API for creating an expert system.

Vidwan, a shell developed at the National Centre for Software Technology, Mumbai in 1993. It enables knowledge encoding in the form of IF-THEN rules.

Development of Expert Systems: General Steps

The process of ES development is iterative. Steps in developing the ES include –

Identify Problem Domain

The problem must be suitable for an expert system to solve it.

Find the experts in task domain for the ES project.

Establish cost-effectiveness of the system.

Design the System

Identify the ES Technology

Know and establish the degree of integration with the other systems and databases.

Realize how the concepts can represent the domain knowledge best.

Develop the Prototype

From Knowledge Base: The knowledge engineer works to –

Acquire domain knowledge from the expert.

Represent it in the form of If-THEN-ELSE rules.

Test and Refine the Prototype

The knowledge engineer uses sample cases to test the prototype for any deficiencies in performance.

End users test the prototypes of the ES.

Develop and Complete the ES

Test and ensure the interaction of the ES with all elements of its environment, including end users, databases, and other information systems.

Document the ES project well.

Train the user to use ES.

Maintain the System

Keep the knowledge base up-to-date by regular review and update.

Cater for new interfaces with other information systems, as those systems evolve.

Benefits of Expert Systems

Availability: They are easily available due to mass production of software.

Less Production Cost: Production cost is reasonable. This makes them affordable.

Speed: They offer great speed. They reduce the amount of work an individual puts in.

Less Error Rate: Error rate is low as compared to human errors.

Reducing Risk: They can work in the environment dangerous to humans.

Steady response: They work steadily without getting motional, tensed or fatigued.